



**Technical University of Munich**

School of Computation, Information and Technology

-- Informatics --

Bachelor's Thesis in Information Systems

**AI-Driven Workflow for Inconsistency  
Remediation in Programming  
Exercises**

Mikhail Khinevich



# **Technical University of Munich**

School of Computation, Information and Technology  
-- Informatics --

Bachelor's Thesis in Information Systems

## **AI-Driven Workflow for Inconsistency Remediation in Programming Exercises**

KI-gesteuerter Workflow zur  
Inkonsistenzbehebung in Programmieraufgaben

|                         |                                              |
|-------------------------|----------------------------------------------|
| <b>Author:</b>          | Mikhail Khinevich                            |
| <b>Supervisor:</b>      | Prof. Dr. Stephan Krusche                    |
| <b>Advisors:</b>        | Dr. Sandro Speth, Felix T.J. Dietrich, M.Sc. |
| <b>Start Date:</b>      | 06.11.2025                                   |
| <b>Submission Date:</b> | 21.04.2026                                   |

I confirm that this Bachelor's thesis is my own work and I have documented all sources and material used.

Munich, 21.04.2026

Mikhail Khinevich

# Transparency in the Use of AI Tools

In preparing this thesis, I used the following AI-powered tools:

**Anthropic AI (Claude Code)** served as a multi-purpose assistant throughout this thesis. For the written document, it assisted with restructuring paragraphs for active voice compliance, suggesting alternative phrasings, and identifying style violations. I reviewed, edited, and verified all text produced with this tool. Claude Code supported literature searches, citation validation, and bibliography management. On the implementation side, Claude Code assisted with reading and explaining existing Artemis codebase context, planning implementation approaches, generating dataset variants for evaluation, conducting preemptive self-reviews of pull requests, and triaging pull request review comments.

**Scite**, **Semantic Scholar** and **Google Scholar** supported the scientific literature research process. These tools enabled structured searches of peer-reviewed publications, retrieval of citation contexts, and validation of referenced claims against their source papers.

**DeepL** assisted with translating the German abstract from the English original and improving the language quality.

I have carefully reviewed all outputs produced by these tools to ensure correctness, coherence, and academic integrity. All code generated by AI tools was guided, reviewed, and verified by me line by line. The intellectual contributions, including the system design decisions, architectural trade-offs, implementation strategies, and evaluation, are entirely my own work.

# Acknowledgements

I would like to thank my advisor Prof. Dr. Stephan Krusche, who welcomed me into this huge TUM Applied EdTech family. Your guidance and support were invaluable throughout this journey.

A big shoutout to my advisors Tobias Wasner, Felix T.J. Dietrich, Dr. Sandro Speth and the whole Hyperion Team: Ailipaer, Khiem, Konstantin and You Sheng, for your patience and for the support you provided, essential feedback and for always being available to answer my questions and review my merge requests.

To my friends and family, who have always been there for me, thank you for your encouragement and understanding, especially during the stressful times. Your support kept me going.

I would especially like to thank my close friends. Thank you for constantly joking around and making fun of me, it made me smile during these difficult times. I also want to thank the luuuunch\_gang from Avelios Medical. Because of you fellas, I had to stay late at the office and write my thesis at night. Thank you for the sleepless nights and your unwavering support.

## Abstract

The Technical University of Munich developed Artemis, an open-source learning management system that hosts multi-artifact programming exercises. Each exercise consists of four interconnected artifacts — problem statement, template, solution and test repositories — that instructors and teaching assistants collaboratively create and maintain. Inconsistencies between these artifacts arise when an instructor modifies one artifact but overlooks the corresponding changes in dependent ones, producing mismatched method names or conflicting type declarations. The Hyperion consistency check within Artemis uses large language models (LLMs) to detect these inconsistencies, but suffers from low precision: the baseline produces nearly one false positive for every two correct detections, forcing instructors to manually verify each reported issue.

This thesis addresses the precision bottleneck and the limitations of the existing detection system through three contributions: (1) It establishes a reproducible benchmarking workflow that connects Artemis with PECV-bench, an evaluation framework for consistency detection, enabling systematic comparison of detection approaches under identical conditions. (2) It extends the evaluation dataset from three Java exercises with 91 variants to 16 exercises with 325 variants across six university courses and multiple programming languages, including Python, C, Assembler, SQL, and Swift. (3) It improves detection precision by introducing a language-aware context filter that removes irrelevant files before analysis and a verification checker that re-evaluates each detected issue to filter false positives.

The verification checker improves precision from 0.531 to 0.819 while maintaining recall at 0.767, raising the  $F_1$  score from 0.633 to 0.793 on the extended dataset. Instructors benefit from a lower false positive rate, which reduces manual review and increases the reliability of the automated analysis. The approach relies on closed-source LLM APIs, which introduce non-determinism and limit reproducibility. Abstract Syntax Trees, Knowledge Graphs, and Graph-based Retrieval-Augmented Generation represent the primary direction for future work targeting the remaining false positives.

## **Zusammenfassung**

Artemis ist ein Open-Source Learning Management System, das an der TUM entwickelt wurde und Multi-Artifact Programming Exercises bereitstellt. Jede Aufgabe besteht aus vier miteinander verbundenen Artifacts — Problem Statement, Template, Solution und Test Repository — die Dozenten und Tutoren gemeinsam erstellen und pflegen. Inkonsistenzen entstehen, wenn ein Dozent ein Artifact ändert, die Anpassungen in abhängigen Artifacts jedoch übersieht, was zu inkonsistente Methodennamen oder widersprüchlichen Typdeklarationen führt. Der Hyperion Consistency Check nutzt Large Language Models zur Erkennung dieser Inkonsistenzen, leidet jedoch unter geringer Precision: Die Baseline erzeugt nahezu einen False Positive auf je zwei korrekte Erkennungen, was Dozenten dazu zwingt, jeden Befund manuell zu überprüfen.

Diese Arbeit adressiert den Precision-Engpass durch drei Beiträge: (1) Der erste Beitrag etabliert einen reproduzierbaren Benchmarking-Workflow, der Artemis mit PECV-bench verbindet und einen systematischen Vergleich von Detection-Ansätzen ermöglicht. (2) Der zweite Beitrag erweitert das Dataset von drei Java-Aufgaben mit 91 Varianten auf 16 Aufgaben mit 325 Varianten aus sechs Kursen und mehreren Programmiersprachen, darunter Python, C, Assembler, SQL und Swift. (3) Der dritte Beitrag verbessert die Precision durch einen sprachbezogenen Context Filter sowie einen Verification Checker, der jeden erkannten Befund erneut bewertet.

Der Verification Checker verbessert die Precision von 0.531 auf 0.819 bei einem Recall von 0.767 und erhöht den  $F_1$ -Score von 0.633 auf 0.793. Dozenten profitieren von einer niedrigeren False-Positive-Rate, was den manuellen Prüfaufwand reduziert und die Zuverlässigkeit der automatisierten Analyse erhöht. Der Ansatz stützt sich auf Closed-Source LLM APIs, was Non-Determinismus einführt und die Reproduzierbarkeit einschränkt. Abstract Syntax Trees, Knowledge Graphs und Graph-based Retrieval-Augmented Generation stellen zukünftige Ansätze zur weiteren Reduzierung von False Positives dar.

# Contents

|                                                                |           |
|----------------------------------------------------------------|-----------|
| <b>1 Introduction</b>                                          | <b>1</b>  |
| 1.1 Problem Area                                               | 1         |
| 1.2 Motivation and Research Gap                                | 2         |
| 1.3 Objectives and Research Questions                          | 4         |
| 1.3.1 Establish an On-Demand Benchmarking Workflow             | 4         |
| 1.3.2 Extend the Consistency Check Dataset                     | 4         |
| 1.3.3 Improve Consistency Issues Detection                     | 5         |
| 1.4 Evaluation Summary                                         | 5         |
| 1.5 Structure of the Thesis                                    | 6         |
| <b>2 Background</b>                                            | <b>7</b>  |
| 2.1 Artemis and Multi-Artifact Programming Exercises           | 7         |
| 2.2 Consistency Issues in Programming Exercises                | 9         |
| 2.3 Large Language Models                                      | 10        |
| 2.4 The Hyperion Consistency Check Pipeline                    | 10        |
| 2.5 PECV-bench and Evaluation Metrics                          | 13        |
| 2.6 Limitations of the Existing System                         | 15        |
| <b>3 Related Work</b>                                          | <b>17</b> |
| 3.1 Consistency Checking in Software Engineering and Education | 17        |
| 3.2 LLM-Based Code Analysis and Verification                   | 18        |
| 3.3 Techniques for Improving LLM Precision                     | 18        |
| 3.4 Research Positioning                                       | 19        |
| <b>4 Requirements</b>                                          | <b>20</b> |
| 4.1 Requirements Elicitation                                   | 20        |
| 4.2 Benchmark Requirements                                     | 21        |
| 4.3 Instructor Requirements                                    | 22        |
| 4.4 Quality Attributes and Constraints                         | 24        |



|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| 4.5      | System Models .....                                    | 25        |
| 4.5.1    | Scenarios .....                                        | 25        |
| 4.5.2    | Domain Model .....                                     | 27        |
| 4.5.3    | Dynamic Model .....                                    | 29        |
| <b>5</b> | <b>Architecture .....</b>                              | <b>30</b> |
| 5.1      | Overview .....                                         | 31        |
| 5.2      | Design Goals .....                                     | 31        |
| 5.3      | Subsystem Decomposition .....                          | 32        |
| 5.3.1    | Hyperion Consistency Check Module .....                | 33        |
| 5.3.2    | Evaluation Workflow .....                              | 34        |
| 5.3.3    | PECV-bench and PECV-bench-dataset .....                | 35        |
| 5.4      | Dataset Structure and Extension .....                  | 36        |
| 5.5      | Persistent Data Management .....                       | 36        |
| 5.6      | Access Control .....                                   | 37        |
| <b>6</b> | <b>Benchmarking Workflow .....</b>                     | <b>37</b> |
| 6.1      | Workflow Overview .....                                | 38        |
| 6.2      | Environment Setup and Configuration .....              | 39        |
| 6.3      | Variant Materialization and Course Preparation .....   | 40        |
| 6.4      | Consistency Check Execution .....                      | 41        |
| 6.5      | Evaluation and Reporting .....                         | 42        |
| 6.6      | Limitations .....                                      | 43        |
| <b>7</b> | <b>Data Creation .....</b>                             | <b>44</b> |
| 7.1      | Dataset Overview .....                                 | 44        |
| 7.2      | Data Collection .....                                  | 46        |
| 7.2.1    | Exercise Selection Criteria .....                      | 46        |
| 7.2.2    | Sources and Contributors .....                         | 46        |
| 7.3      | Variant Creation and Annotation .....                  | 47        |
| 7.3.1    | Remediation of Pre-Existing Inconsistencies .....      | 48        |
| 7.3.2    | Inconsistency Injection .....                          | 48        |
| 7.3.3    | Gold-Standard Annotation and Manual Verification ..... | 50        |

|                                                        |           |
|--------------------------------------------------------|-----------|
| 7.3.4 Justification and Bias Controls .....            | 50        |
| 7.4 Limitations .....                                  | 51        |
| <b>8 Consistency Check Approaches .....</b>            | <b>52</b> |
| 8.1 Evaluation Design .....                            | 52        |
| 8.1.1 Benchmark Configuration .....                    | 52        |
| 8.1.2 Metrics and Evaluation Criteria .....            | 52        |
| 8.1.3 Research Questions .....                         | 53        |
| 8.2 Baseline Approach .....                            | 54        |
| 8.3 Language-Aware Context Filter .....                | 56        |
| 8.4 Verification Checker .....                         | 60        |
| 8.5 Unified Consistency Checker .....                  | 63        |
| 8.6 Future Approaches .....                            | 66        |
| 8.6.1 Abstract Syntax Tree Representation .....        | 67        |
| 8.6.2 Knowledge Graph Representation .....             | 68        |
| 8.6.3 Graph-Based Retrieval-Augmented Generation ..... | 69        |
| 8.7 Overall Discussion .....                           | 70        |
| 8.8 Threats to Validity .....                          | 72        |
| <b>9 Conclusions &amp; Future Work .....</b>           | <b>73</b> |
| 9.1 Summary .....                                      | 73        |
| 9.2 Discussion .....                                   | 74        |
| 9.3 Status .....                                       | 75        |
| 9.3.1 Realized Goals .....                             | 76        |
| 9.3.2 Open Goals .....                                 | 77        |
| 9.4 Future Work .....                                  | 78        |
| <b>List of Figures .....</b>                           | <b>79</b> |
| <b>List of Tables .....</b>                            | <b>82</b> |
| <b>Appendix A: Supplementary Material .....</b>        | <b>84</b> |
| <b>Bibliography .....</b>                              | <b>85</b> |

# 1 Introduction

Computer science enrollment at European universities has grown steadily over the past decade, placing increasing pressure on instructors to deliver high-quality programming education at scale. Learning Management Systems (LMS) emerged to address this challenge: they automate assessment processes and enable instructors to manage large groups of students without a corresponding increase in manual work [KS18]. The Technical University of Munich (TUM) developed Artemis, an open-source LMS for interactive learning and automatic assessment in programming courses, as part of this evolution [KS18].

Artemis supports multi-artifact programming exercises. Each exercise consists of four interconnected artifacts — problem statement, template, solution and test repositories — that instructors and teaching assistants collaboratively create and update throughout a course lifecycle. Maintaining consistency across all four artifacts is an ongoing challenge, since a change to one artifact can silently invalidate the others. The Hyperion service extends Artemis with an LLM-based consistency check that analyzes exercise artifacts and detects consistency issues [Die+25].

## 1.1 Problem Area

A common example illustrates how consistency issues arise in multi-artifact programming exercises: the problem statement instructs students to implement a method named `calculateAverage`, while the template repository contains a placeholder for a method named `computeMean` [Die+25]. Students who encounter this mismatch must decide whether to follow the written specification or the provided code structure, diverting cognitive resources away from the intended learning objective. These inconsistencies compromise the fairness of automated grading when different students resolve the contradiction differently.

Instructors must identify and resolve these inconsistencies across all exercise materials. The detection process remains manual, time-consuming, and error-prone — particularly in large courses where multiple teaching assistants divide exercise creation and maintenance among themselves. Each modification to one

artifact requires a manual review of all dependent artifacts. Automated detection through LLMs addresses this manual effort but introduces a different problem: the existing system produces a high number of false positives, which forces instructors to manually verify each reported issue and drives them back to manual Quality Assurance (QA). The high volume of false positives undermines the efficiency gain that the automation provides and erodes instructor trust in the tool.

## 1.2 Motivation and Research Gap

High-precision consistency checking enables reliable, large-scale exercise quality assurance that directly improves the conditions for student learning. Mayer’s Coherence Principle [MM03], Biggs’ Constructive Alignment [Big96], and Sweller’s Cognitive Load Theory [Swe88, Swe94] each require that instructional materials present a single coherent representation of the task. A detection system that reliably identifies genuine inconsistencies — and suppresses spurious reports — gives instructors the confidence to act on automated findings without additional manual investigation, which scales exercise QA to courses with hundreds of students and multiple concurrent exercise authors.

The practical value of an automated detection system therefore depends on its precision. A reliable tool with high precision allows instructors to act on each reported issue without additional investigation, reducing the per-exercise review cycle from a full manual artifact scan to a targeted correction of confirmed findings. This shift from manual to targeted review makes consistent exercise authoring sustainable at the scale of large university courses, where individual instructor capacity cannot grow proportionally with enrollment.

Dietrich et al. [Die+25] establish the first empirical baseline for this task and demonstrate that LLM-based detection is feasible — Section 2.5 reports the full per-model results. The baseline achieves high recall but leaves precision below the threshold at which instructors can act on reports without additional manual verification. A Pearson correlation test between prompt token count and per-variant  $F_1$  score across all four evaluated models reveals a weak and statistically insignificant correlation at the  $p < 0.05$  level, as Figure 1 shows. This analysis rules

out input length as the primary limiting factor and points toward a structural cause: the pipeline represents all exercise artifacts as a single block of unstructured text, requiring the LLM to infer cross-artifact relationships implicitly.

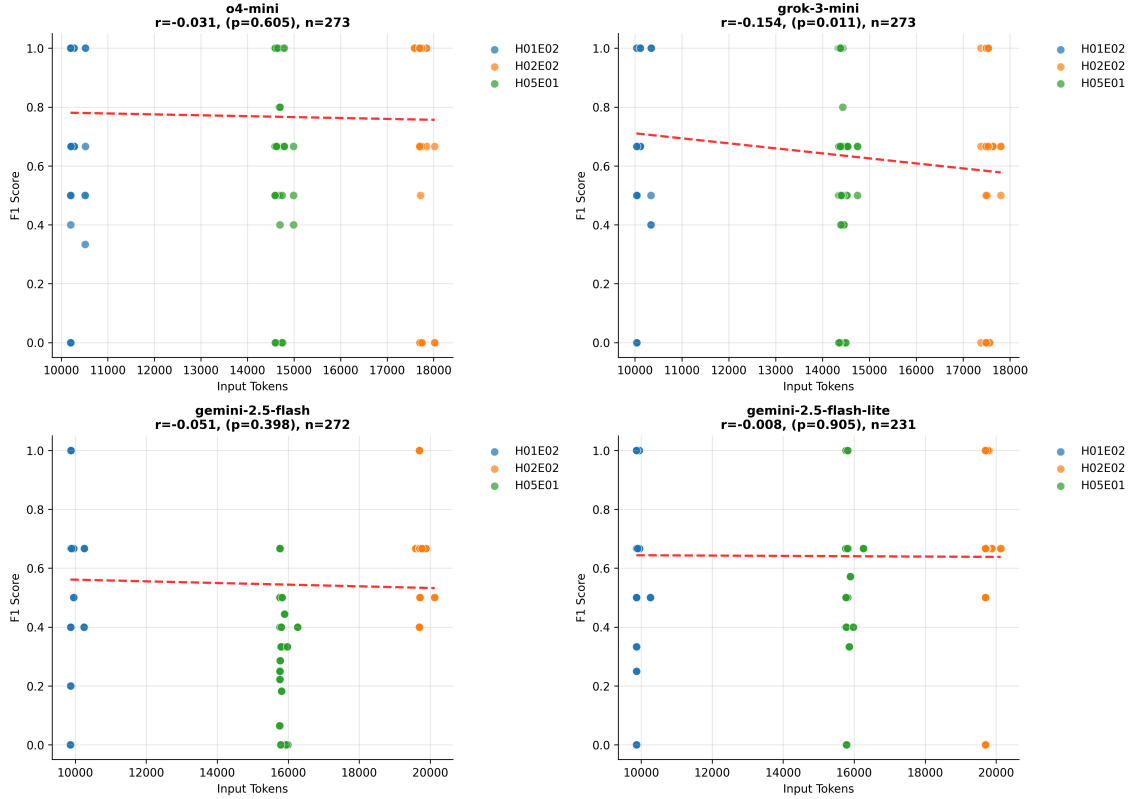


Figure 1: Correlation between input tokens and per-variant  $F_1$  score across all four evaluated reasoning models (o4-mini, Grok 3 Mini, Gemini 2.5 Flash, Gemini 2.5 Flash Lite) on the dataset with 91 perturbed variants. Each point represents a single variant evaluation.

Consistency checking between natural language specifications and source code is a known challenge in software engineering. Traditional formal approaches require complete formal specifications that educational settings rarely provide [Li+16]. LLMs close this cross-modal gap by reasoning over natural language and source code simultaneously, and chain-of-thought prompting improves reliability on multi-step inference tasks [Wei+22]. The precision value reported by Dietrich et al. [Die+25], however, shows that single-pass LLM detection without a verification stage leaves too many false positives unfiltered. No existing work introduces a

dedicated post-detection filtering stage or evaluates consistency detection across programming languages beyond Java.

### 1.3 Objectives and Research Questions

This gap produces the central research question of the thesis: which approaches reduce the false positive rate of LLM-based consistency detection for multi-artifact programming exercises while preserving recall, and how do the improvements generalize across programming languages? Three objectives operationalize this question.

1. Establish an On-Demand Benchmarking Workflow (Section 1.3.1)
2. Extend the Consistency Check Dataset (Section 1.3.2)
3. Improve Consistency Issues Detection (Section 1.3.3)

#### 1.3.1 Establish an On-Demand Benchmarking Workflow

The first objective creates an integrated benchmarking workflow that connects Artemis<sup>1</sup> with the PECV-bench<sup>2</sup> evaluation framework, enabling systematic and reproducible evaluation of any consistency check approach against the established baseline. The benchmarking workflow serves as the foundation for all subsequent objectives. The pipeline evaluates each new detection approach or prompt modification under identical conditions, ensuring that all results remain directly comparable. The workflow produces per-variant, per-category, and per-model performance breakdowns. Each evaluation run records the exact state of the Hyperion codebase, the model configuration, and the dataset version, which enables any developer to reproduce a specific result.

#### 1.3.2 Extend the Consistency Check Dataset

The second objective extends the PECV-bench dataset beyond the original three Java exercises with 91 variants. The original dataset from Dietrich et al. [Die+25] provides a starting point but covers a limited scope of exercises and a single programming language. The extension adds real exercises from six TUM courses

---

<sup>1</sup><https://github.com/lslintum/Artemis>

<sup>2</sup><https://github.com/lslintum/PECV-bench>

across multiple programming languages — including Python, C, Assembler, SQL, and Swift — expanding the dataset to better reflect the diversity of issues that arise in practice.

### **1.3.3 Improve Consistency Issues Detection**

The third objective reduces the false positive rate of the consistency check system while maintaining its high recall. This objective pursues two complementary improvements to the detection pipeline: a language-aware context filter and a verification checker. The filter extends the existing Java-only file filtering to support programming languages most used in TUM courses, which the extended dataset covers.

The verification checker introduces a third analysis step that runs sequentially after the existing structural and semantic checkers. This step receives the issues that the first two parallel checkers detected and re-evaluates each one against the exercise artifacts, removing false positives before the report reaches the instructor. The design targets the precision bottleneck directly: rather than modifying how the system detects issues, it adds a dedicated filtering stage that validates each detection.

Chapter 9 discusses a future direction that supplements this approach — enhancing the context representation through Abstract Syntax Trees, Knowledge Graphs, and Graph-based Retrieval-Augmented Generation to provide the LLM with structured artifact relationships.

## **1.4 Evaluation Summary**

The verification checker raises precision from 0.531 to 0.819 on the extended dataset while maintaining recall at 0.767, improving the  $F_1$  score from 0.633 to 0.793. The language-aware context filter reduces irrelevant content in the rendered context across all six programming languages in the extended dataset. The combination of both extensions reduces false positives from 230 to 56 on the extended dataset, which cuts the workload by 75% compared to the first empirical baseline.

Two limitations constrain the scope of these results. The detection pipeline relies on closed-source LLM Application Programming Interfaces (APIs) whose outputs are non-deterministic: identical inputs produce different issue reports across runs, which introduces variance into all reported metrics. The pipeline represents exercise artifacts as flat unstructured text, leaving cross-artifact relationships implicit. This unstructured representation remains the root cause of the remaining false positives and motivates the structured context representations that this thesis identifies as the primary direction for future work.

## 1.5 Structure of the Thesis

Chapter 2 introduces the foundational concepts that underpin this work: the Artemis platform, the consistency issue ontology, LLMs, the Hyperion detection pipeline, and the PECV-bench evaluation framework. Chapter 3 surveys related work on consistency checking in software engineering and education, LLM-based code analysis, and techniques for improving LLM output precision.

Chapter 4 derives the requirements for the proposed system from the limitations of the existing system, specifying functional requirements, quality attributes, and constraints. Chapter 5 maps these requirements to the system architecture, describing the subsystem decomposition and the design of each contribution.

Chapter 6 describes the benchmarking workflow in detail, covering the end-to-end pipeline from variant materialization to result reporting. Chapter 7 describes the dataset construction process, including data collection, annotation methodology, and variant injection across six programming languages. Chapter 8 presents and evaluates the consistency check approaches — the baseline, the language-aware context filter, and the verification checker — and discusses results per approach. Chapter 9 closes the thesis with a summary of realized and open goals, a discussion of limitations, and directions for future work.



## 2 Background

This chapter introduces the concepts and technologies necessary to understand the context and proposed solution of this thesis. It focuses on the Artemis platform and its multi-artifact exercise structure, consistency issues in programming exercises and their taxonomy, LLMs and their role in automated analysis, the existing Hyperion consistency check pipeline, and the PECV-bench evaluation framework. The final section identifies the limitations of the existing system. These topics form the conceptual basis for the requirements analysis, architecture, and evaluation chapters that follow.

### 2.1 Artemis and Multi-Artifact Programming Exercises

Artemis builds on version control and continuous integration to support a wide range of exercise types, including programming, quiz, text, modeling, and file upload exercises [KS18]. The platform serves large-scale courses with thousands of participants and provides instructors with tools to manage exercises, track student progress, and ensure assessment quality. Artemis centrally supports multi-artifact programming exercises, which develop practical programming skills in students.

Programming exercises in Artemis consist of four interconnected artifacts that together define a complete task. The problem statement describes the task in natural language and specifies the requirements that guide student implementation. The template repository provides starter code that gives students a defined entry point for their work. The solution repository contains a reference implementation that fulfills the problem statement. The test repository holds automated test cases that evaluate the correctness of student submissions against predefined criteria [KS18]. Instructors and teaching assistants collaboratively create and maintain these artifacts throughout a course lifecycle.

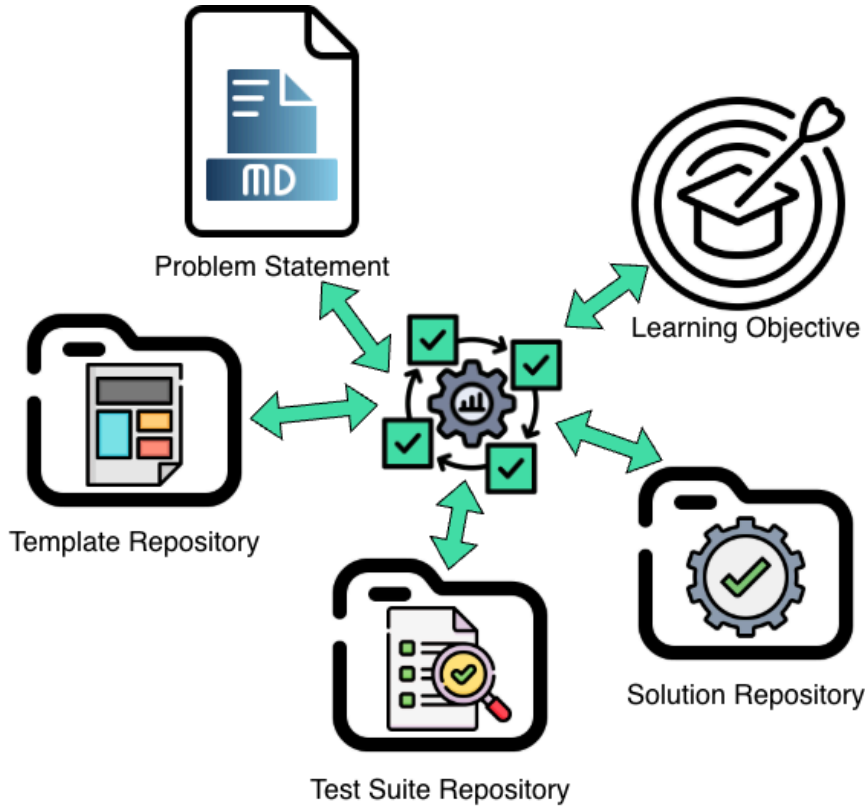


Figure 2: Dependency structure of the four exercise artifacts in Artemis.

These four artifacts form a directed dependency structure, as Figure 2 illustrates. A change to one artifact can invalidate the assumptions of dependent artifacts — for example, renaming a method in the problem statement requires corresponding updates in the template, solution, and test suite. Students see only the problem statement and the template repository during the exercise phase. The solution repository and the test repository remain accessible only to instructors and serve as the basis for correctness evaluation [Die+25].

This multi-artifact structure creates a dual persistence boundary. Exercise metadata such as titles, deadlines, and grading configurations resides in the relational database, while exercise content resides in git repositories. A single logical exercise update may affect the database, one or more git repositories, or both. Any system that analyzes exercise consistency must access information from both persistence layers to obtain a complete view of the exercise state.

## 2.2 Consistency Issues in Programming Exercises

Dietrich et al. [Die+25] define a consistency issue as “an unintentional violation of instructional coherence that creates extraneous cognitive load by presenting contradictory information about the same educational element”. Inconsistencies between exercise artifacts force students to reconcile conflicting representations of the same concept rather than focusing on the intended learning objective.

The ontology-based model proposed by Dietrich et al. [Die+25] captures the structure, dependencies, and inconsistency types across heterogeneous exercise artifacts. The ontology defines five theoretically grounded consistency categories, of which the current system instantiates and evaluates two — Structural and Semantic. Structural inconsistencies occur when overlapping elements across different artifacts make conflicting assertions that cannot simultaneously hold. Semantic inconsistencies arise when the same knowledge element carries different names across artifacts, creating cognitive mapping barriers for students.

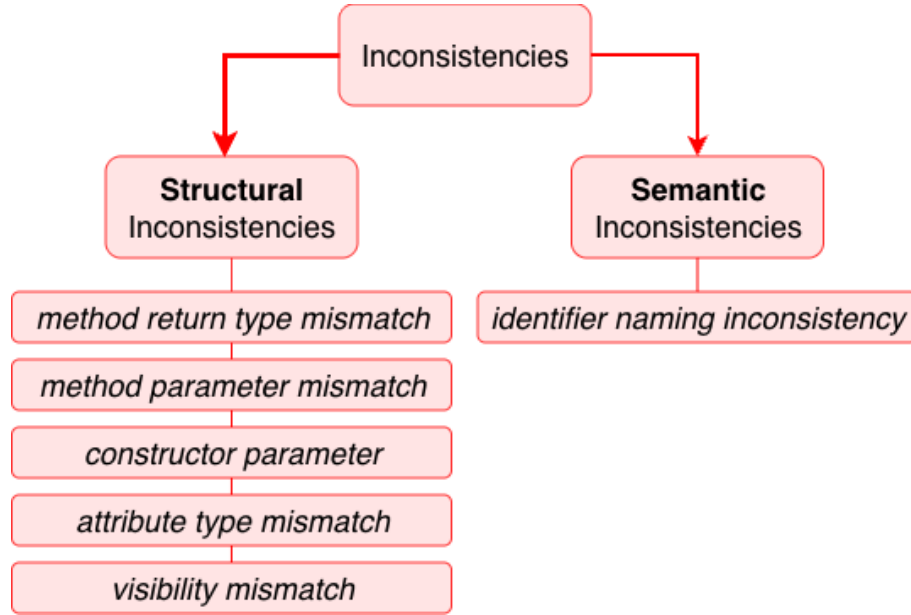


Figure 3: Consistency issue ontology for multi-artifact programming exercises. Each subcategory defines a specific type of inconsistency between the problem statement, template repository, solution repository and test repository [Die+25].

The Structural category covers five subcategories, as Figure 3 illustrates, and the Semantic category covers one subcategory [Die+25]. These six subcategories form the basis for the detection system that this thesis evaluates.

## 2.3 Large Language Models

LLMs are deep neural networks built on the transformer architecture that learn statistical patterns from large amounts of text and source code during pre-training. These models accept a sequence of input tokens — a prompt — and generate a sequence of output tokens that represents a response. The consistency check task requires cross-modal reasoning over natural language specifications in the problem statement and formal code structures in the template and solution repositories simultaneously. Structured output generation constrains the model to produce responses that conform to a predefined JSON schema, ensuring that each detected issue includes a category, severity, description, and artifact locations in a machine-parseable format.

These capabilities come with limitations that directly affect detection precision. LLMs can hallucinate — they may report inconsistencies that do not exist in the input artifacts, generating false positives. The models exhibit sensitivity to prompt design: small changes in instruction phrasing, example selection, or output format can produce different detection results on identical input. Both limitations are central to the precision problem that this thesis addresses.

## 2.4 The Hyperion Consistency Check Pipeline

The Hyperion service within Artemis provides LLM-based assistance for exercise QA. The consistency check pipeline accepts a programming exercise, analyzes its artifacts, and returns a structured report of detected issues. An instructor triggers this check, and the system performs the full analysis without requiring manual configuration of which artifacts to compare or which issue types to search for [Die+25].

The pipeline executes three sequential steps: context rendering, prompt construction, and LLM-based analysis. The context rendering step transforms the

heterogeneous exercise artifacts into a unified text representation suitable for LLM processing. A language-specific filter restricts the rendered files to those relevant for the exercise's programming language. The baseline pipeline registers a single filtering strategy for Java that includes only `.java` files under `src/` directories and excludes hidden files and files exceeding 50 kilobytes.

The renderer produces a deterministic text snapshot that combines three sections: the problem statement rendered as a pseudo-file with line numbers, the template repository with a file tree overview followed by line-numbered source files, and the solution repository in the same format. Figure 4 shows a concrete example of this rendered output, illustrating how the three artifact sections appear as a single unified block that the LLM receives as input [Die+25].

```
==== Problem Statement ====
problem_statement.md
1 | # H01E02 - Lectures
2 | Implement a class representing a lecture.
3 | 1. Add attributes: `lectureName`, `numberOfInscribedStudents`
4 | 2. Create a constructor that initializes every attribute.
==== Template Repository ====
template_repository
`-- src
   |-- com
   |   |-- example
   |       |-- exercise
   |           |-- Lecture.java
   |           |-- Main.java
template_repository/src/com/example/exercise/Lecture.java
1 | package com.example.exercise;
2 | public class Lecture {
3 |     // TODO 1.1: Add attributes from the UML diagram
4 |     // TODO 1.2: Add the constructor
5 | }
```

Figure 4: Simplified example of the context rendering output format. The rendered context combines the problem statement as a pseudo-file with line numbers and the template repository with a file tree overview followed by line-numbered source files [Die+25].

The prompt construction step builds the prompts that the subsequent analysis step uses. The baseline pipeline builds two separate prompts — one for structural analysis and one for semantic analysis — but the pipeline design treats the number and composition of prompts as a pluggable decision. Chapter 8 evaluates alternative configurations, including a single combined prompt that covers both structural and semantic inconsistencies.

The structural prompt instructs the LLM to detect five structural mismatch subcategories. The prompt distinguishes between pedagogical gaps that the exercise design introduces intentionally — such as missing method bodies in the template — and structural contradictions that represent genuine errors. The semantic prompt instructs the LLM to detect semantic mismatch subcategories using a four-step analysis framework: conceptual entity identification, cognitive mapping assessment, contextual validation, and inconsistency classification. Both prompts embed references to cognitive load theory and include few-shot examples that distinguish true positives from false positives [Die+25].

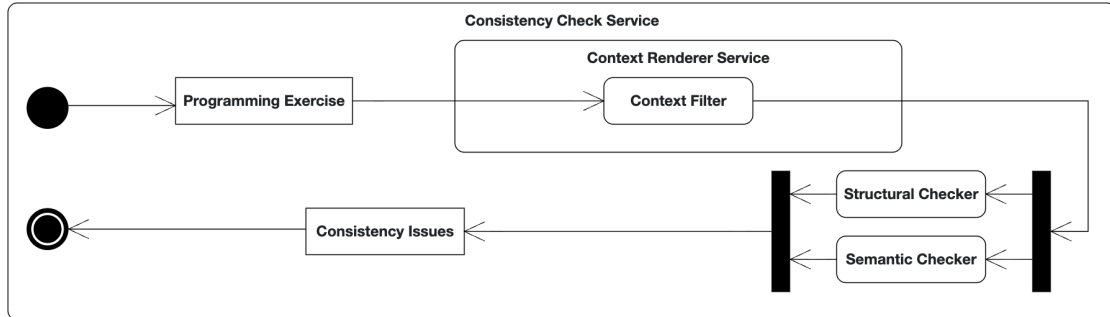


Figure 5: Activity diagram of the existing Hyperion consistency check pipeline.

The LLM-based analysis step executes the configured checkers and aggregates their results, as Figure 5 illustrates. The baseline pipeline runs the structural and semantic checkers in parallel and merges their findings after both complete. Each checker produces consistency issues that conform to the ontology defined in Section 2.2: each issue carries a severity level, a category, a textual description, a suggested fix, and artifact locations specifying file paths and line ranges [Die+25].

This structured output format enables downstream tools — such as the PECV-bench evaluation framework that the next section describes — to compare detected issues against gold-standard annotations at the line level. When a checker fails due to a network error, timeout, or unparseable response, the pipeline returns an empty list for that checker rather than aborting the entire analysis.

## 2.5 PECV-bench and Evaluation Metrics

Dietrich et al. [Die+25] publish the PECV-bench<sup>3</sup> (Programming Exercise Consistency Verification benchmark) as a reproducibility package alongside their research. The package contains a dataset of annotated exercise variants, a command-line interface for managing variants and executing benchmark runs, a reference LLM pipeline that reproduces the published results, and evaluation scripts that align predictions with gold-standard annotations and aggregate metrics. PECV-bench serves two purposes: it enables reproducible evaluation of consistency detection approaches, and it provides a common baseline against which developers can compare new approaches under identical conditions.

The dataset contains three Java programming exercises of varying complexity from Artemis courses at TUM, with 91 perturbed variants across these exercises (30, 29, and 32 respectively) and 93 annotated consistency issues [Die+25]. Each variant in the dataset starts from a clean base exercise that contains no inconsistencies. A git-style unified diff patch injects exactly one consistency issue into one or more artifacts of the exercise. The PECV-bench CLI materializes a variant by copying the base artifacts into the variant directory and applying the corresponding patch file. Each variant includes a gold-standard JSON annotation file that specifies the expected issues with their category, severity, affected artifact locations, and line ranges. This structure enables fully automated comparison between the issues that a detection system reports and the issues that the gold standard defines.

---

<sup>3</sup><https://github.com/lslintum/PECV-bench>

The evaluation framework computes detection-level metrics from three basic counts. A true positive (TP) occurs when a detected issue matches a gold-standard issue. A false positive (FP) occurs when a detected issue has no corresponding gold-standard counterpart. A false negative (FN) occurs when the system fails to detect a gold-standard issue. Precision measures the share of detected issues that are correct — a high precision value indicates that the system rarely reports non-existent issues. Recall measures the share of gold-standard issues that the detector recovers — a high recall value indicates that the system rarely misses real issues. The  $F_1$  score combines precision and recall through their harmonic mean, producing a single summary metric.

The evaluation framework complements detection-level metrics with localization metrics that measure how precisely the detector identifies the affected lines of code. Span  $F_1$  uses the Dice coefficient to quantify the overlap between predicted and gold-standard line spans:  $\text{Span } F_1 = \frac{2|A \cap B|}{|A| + |B|}$ , where  $A$  and  $B$  are the sets of lines marked by the detector and the gold standard respectively. The Intersection over Union (IoU) metric, the Jaccard index, applies a stricter overlap measure:  $\text{IoU} = \frac{|A \cap B|}{|A \cup B|}$ . IoU penalizes predictions that cover large spans beyond the gold-standard lines more heavily than Span  $F_1$ . Both metrics reward predictions that capture the relevant code lines even when the exact boundaries differ by a few statements [Die+25].

| Model                    | N<br>runs | TP  | FP  | FN | Precision | Recall | $F_1$       | Span<br>$F_1$ | IoU  | Time<br>(s) | Cost<br>(\$) |
|--------------------------|-----------|-----|-----|----|-----------|--------|-------------|---------------|------|-------------|--------------|
| GPT o4-mini              | 3         | 254 | 148 | 25 | 0.63      | 0.91   | <b>0.75</b> | 0.68          | 0.57 | 32.96       | 0.034        |
| Grok 3 Mini              | 3         | 233 | 222 | 46 | 0.51      | 0.84   | 0.63        | 0.64          | 0.53 | 14.31       | 0.006        |
| Gemini 2.5 Flash         | 3         | 263 | 623 | 15 | 0.30      | 0.95   | 0.45        | 0.60          | 0.47 | 26.38       | 0.024        |
| Gemini 2.5 Flash<br>Lite | 3         | 216 | 288 | 21 | 0.43      | 0.91   | 0.58        | 0.59          | 0.49 | 16.97       | 0.006        |

Table 1: Baseline results across four reasoning models on the PECV-bench dataset with 91 perturbed variants containing 93 annotated issues. The highlighted row (GPT o4-mini) achieves the highest  $F_1$  score of 0.75 [Die+25].



The baseline evaluation benchmarks four reasoning models at medium reasoning effort: OpenAI o4-mini, xAI Grok 3 Mini, Google Gemini 2.5 Flash, and Google Gemini 2.5 Flash Lite. Each model executes three runs on every perturbed variant, yielding 1,049 total runs after accounting for failures [Die+25]. The o4-mini configuration achieves the highest  $F_1$  score of 0.75, with a precision of 0.63 and a recall of 0.91. These baseline results establish the reference point against which this thesis evaluates all proposed improvements, and the following section examines where the existing system falls short.

## 2.6 Limitations of the Existing System

The baseline results in Section 2.5 establish two separate reference points for the detection pipeline. PECV-bench provides a standalone Python implementation of the consistency check that runs outside Artemis and serves as the reproducible reference. The Artemis Hyperion consistency check implements the same detection logic inside Artemis using Spring AI<sup>4</sup>, a framework that provides a unified abstraction layer for connecting to different LLM providers. Both pipelines execute the same structural and semantic checker logic against the same dataset, so the only source of difference between their results is LLM non-determinism — identical inputs can produce different outputs across runs. Table 2 shows both configurations evaluated on the same 91-variant dataset at medium reasoning effort.

---

<sup>4</sup><https://spring.io/projects/spring-ai>

| Benchmark      | Model                     | N<br>runs | TP  | FP  | FN | Precision | Recall | F <sub>1</sub> | Span<br>F <sub>1</sub> | IoU   |
|----------------|---------------------------|-----------|-----|-----|----|-----------|--------|----------------|------------------------|-------|
| pecv-reference | gpt-5-<br>mini,<br>medium | 3         | 262 | 197 | 17 | 0.571     | 0.939  | 0.710          | 0.433                  | 0.308 |
| artemis-HEAD   | gpt-5-<br>mini,<br>medium | 3         | 267 | 214 | 12 | 0.555     | 0.957  | 0.703          | 0.431                  | 0.309 |

Table 2: Comparison of the PECV-bench reference pipeline and the Artemis Hyperion consistency check. The Artemis-HEAD pipeline runs on the develop branch (commit [66c9e6b<sup>5</sup>](https://github.com/ls1intum/Artemis/commit/66c9e6b5)). Both configurations use the `azure-openai-gpt-5-mini` model and original dataset [Die+25].

The Artemis results required a manual connection between both systems: the benchmarking workflow developed in this thesis uploaded the PECV-bench variants to a running Artemis instance via its REST API and collected the consistency check responses for evaluation. Chapter 6 explains these steps in detail.

The results in Table 2 reveal three limitations that motivate the three objectives of this thesis. The first limitation concerns detection precision: both configurations achieve a precision near 0.55, meaning that nearly one in two reported issues does not correspond to a real inconsistency. The system provides no mechanism to verify or filter detected issues before presenting them to the instructor. The second limitation concerns dataset scope: the dataset covers only Java exercises from a single course, which prevents evaluation of detection performance across different programming languages. The third limitation concerns evaluation infrastructure: the benchmark framework and the production system operate as separate, disconnected tools — developers cannot evaluate changes to the Artemis pipeline against the benchmark dataset in a reproducible manner.

These three limitations — low precision with missing verification, limited dataset scope, and disconnected evaluation infrastructure — directly motivate the three objectives that Chapter 4 derives as requirements for the proposed system.

---

<sup>5</sup><https://github.com/ls1intum/Artemis/commit/66c9e6b98d>

## 3 Related Work

This chapter surveys work in three areas: consistency checking in software engineering and education, LLM-based code analysis, and techniques for improving LLM precision. Section 3.4 maps identified gaps to the contributions of this thesis.

### 3.1 Consistency Checking in Software Engineering and Education

Researchers have studied identifier inconsistency as a specific subproblem within software engineering. Nuseibeh frames inconsistency as an inevitable artifact of collaborative development and argues that systems should detect and track inconsistencies rather than prevent them outright [Nus96]. Spanoudakis and Zisman [SZ01] survey the field and decompose inconsistency management into six activities — detection, diagnosis, handling, tracking, overlap detection, and policy application — identifying detection and handling as the least mature. Göknil et al. [GKB11] operationalize detection by defining formal semantics for trace relations between requirements artifacts and using first-order logic to identify contradictions and infer implicit dependencies. Li et al. [Li+16] quantify the human cost: false-negative leakage rates grow with artifact complexity, and conflict resolution consumes a disproportionate share of development time. These approaches require artifacts to conform to a formal specification language — a prerequisite that educational exercise materials do not satisfy.

Within software engineering, identifier inconsistency has been studied as a specific subproblem. Kim and Kim [KK16] build a code dictionary from identifier tokens and detect naming inconsistencies by comparing identifiers across source files, achieving high precision without requiring formal specifications. Their work is directly related to the identifier naming inconsistency category in the Hyperion ontology, but it targets single-language codebases and does not reason over natural language descriptions.

In programming education, automated assessment research addresses student submissions rather than exercise material quality. Paiva et al. [PFL23] survey

two decades of automated assessment tools and identify feedback generation as the primary open problem; exercise material correctness has received little systematic attention. Kasneci et al. [Kas+23] survey the opportunities and risks of LLMs in education and identify exercise authoring support as one underexplored application area. Dietrich et al. [Die+25] are the first to formulate multi-artifact consistency as a verifiable property of programming exercises and evaluate LLM-based detection against an annotated benchmark. No prior work evaluates consistency detection across multiple programming languages, and no existing system filters false positives before reporting to instructors.

### 3.2 LLM-Based Code Analysis and Verification

LLMs have been applied to a range of code analysis tasks. Zhao et al. [Zha+24] show that reasoning-capable models consistently outperform instruction-following models on tasks requiring multi-step inference about code semantics. Hu et al. [HWW25] demonstrate that enriching the prompt with structured semantic context improves LLM code review performance more than adding surrounding code context.

Hybrid pipelines that combine LLMs with complementary analysis methods achieve better precision than either alone. Birillo et al. [Bir+24] show that pairing LLMs with static analysis for hint generation in programming tasks substantially reduces hallucinated suggestions compared to LLM-only generation. Liu et al. [LZW25] extend this principle with PMDetector, a three-stage pipeline for vulnerability detection that applies static taint analysis, LLM filtering, and static validation in sequence, achieving 88% precision and 90% recall — significantly outperforming either stage alone. This detect-then-verify architecture directly motivates the verification checker introduced in this thesis.

### 3.3 Techniques for Improving LLM Precision

Prompt engineering techniques reduce error rates in multi-step LLM reasoning. Wei et al. [Wei+22] show that chain-of-thought prompting significantly improves accuracy on arithmetic and commonsense benchmarks by eliciting intermediate

reasoning steps. Wang et al. [Wan+23] introduce self-consistency: sampling multiple diverse reasoning paths and selecting by majority vote reduces variance from any single chain. Weng et al. [Wen+23] extend this with self-verification, where the model re-evaluates its conclusion by solving the original problem with the conclusion as a given; their backward verification reduces errors beyond self-consistency alone.

Structured context representations improve code-related LLM tasks. Wang et al. [Wan+20] show that augmenting Abstract Syntax Trees with control- and data-flow edges substantially improves code clone detection over token-based representations. Knowledge graphs provide a complementary form of structured context: Hogan et al. [Hog+21] survey the field and show that relational graph structures support reasoning over entity relationships not explicitly stated in text. Pan et al. [PC24] demonstrate that grounding LLM reasoning in knowledge graph structure reduces hallucination on multi-hop inference tasks. Zhang et al. [ZC24] introduce GraphRAG, which retrieves relational subgraphs rather than flat text chunks, improving performance on cross-document reasoning tasks.

### 3.4 Research Positioning

Prior work applies none of these techniques to multi-artifact consistency detection in programming exercises. Three gaps emerge. First, formal consistency checking requires structured artifacts that educational exercises lack, and existing identifier inconsistency detection targets single-language codebases without natural language reasoning. Second, automated assessment research focuses on student submissions; no prior system applies a post-detection filtering stage to multi-artifact exercise consistency. Third, hybrid LLM pipelines, chain-of-thought prompting, self-verification, and structured context representations exist as independent contributions that no prior work applies to this domain.

These gaps converge on a single open question: how can LLM-based consistency detection for multi-artifact programming exercises achieve higher precision without sacrificing the recall that the baseline already demonstrates, and whether the resulting improvements generalize across programming languages? The first

two gaps define the scope of this thesis. The third gap — structured context representations as a mechanism for reducing the remaining false positives — remains open and defines the primary direction for future work.

## 4 Requirements

This chapter describes the requirements of the proposed system at the level of the application domain. Following the requirements analysis perspective of Bruegge and Dutoit [BD09], it focuses on the purpose of the system, the involved domain concepts, and the expected behavior from the viewpoint of its users. The requirements analysis centers on consistency detection for multi-artifact programming exercises in Artemis and on the improvements that the proposed system must provide within this process. It addresses detection precision, false positive filtering, multi-language support, dataset extension, and reproducible evaluation infrastructure. The following sections elicit the requirements from empirical and stakeholder sources, specify the functional requirements grouped by actor type, define quality attributes and constraints, and introduce the system models that structure the problem domain [BD09].

### 4.1 Requirements Elicitation

The requirements for the proposed system emerge from two complementary sources: an empirical analysis of the existing system and an analysis of stakeholder needs. Together these sources establish what the system must achieve and for whom [BD09]. The empirical foundation comes from the limitations identified in Section 2.6. Those four shortcomings — low precision, missing verification, limited dataset scope, and disconnected evaluation infrastructure — translate directly into the three objectives of this thesis and define the scope of the functional requirements in Section 4.2 and Section 4.3. The stakeholder analysis identifies two groups whose needs shape the requirements.

Instructors use the consistency check during exercise creation and maintenance. Their primary concern is precision: a report that contains false positives reproduces the manual verification workload the system aims to eliminate. Instruc-

tors require actionable reports with precise artifact locations, clear descriptions, and severity levels that allow them to act on findings without additional investigation.

Developers extend and evaluate the detection pipeline. Their primary concern is reproducibility and comparability: they need a benchmarking infrastructure that evaluates any approach under identical conditions and produces directly comparable results. The infrastructure must also support independent development of new approaches and programming language strategies without requiring changes to the existing pipeline.

During dataset collection for this thesis, instructors who contributed exercises to the extended dataset described exercise QA as a recurring problem — reviewing artifacts for inconsistencies before each course iteration consumes time that instructors would rather spend on pedagogical improvements. The prospect of a reliable automated tool that reduces these issues provided a clear adoption signal and reinforced the precision-first priority that the empirical analysis already indicated.

## 4.2 Benchmark Requirements

The benchmark requirements specify the capabilities the proposed system must provide to support reproducible evaluation of consistency detection approaches. These requirements address the disconnected evaluation infrastructure and limited dataset scope that Section 2.6 establishes, and the needs that stakeholder analysis surfaces: reproducible benchmarking, multi-language dataset coverage, and protection of confidential exercise materials [BD09].

**FR1 Analyze Detection Performance:** The system shall analyze detection performance by producing per-variant evaluation reports and supporting analysis of the relationship between input characteristics and detection accuracy.

**FR2 Connect Benchmark to Production System:** The system shall execute the benchmark evaluation against the same consistency check implemen-

tation that the production system uses, ensuring that evaluation results reflect actual production behavior.

**FR3 Execute Benchmark Evaluation:** The system shall run the consistency check against all variants in the dataset and compute detection metrics including precision, recall,  $F_1$ , Span  $F_1$ , and IoU.

**FR4 Compare Approaches Against Baseline:** The system shall evaluate different detection configurations under identical conditions and produce directly comparable result reports.

**FR5 Manage Benchmark Dataset:** The system shall create, store, and materialize exercise variants with injected inconsistencies and corresponding gold-standard annotations that the thesis author verifies against real university exercises.

**FR6 Support Multiple Programming Languages in Dataset:** The dataset shall include exercises written in different programming languages to eliminate the single-language bias of the original dataset.

**FR7 Separate Public and Confidential Exercise Components:** For university exercises currently in use, the system shall store problem statements and template repositories as publicly accessible dataset components and store test suites along with solution repositories in private repositories that only authorized approach developers can access.

### 4.3 Instructor Requirements

The instructor requirements specify the capabilities the proposed system must provide to support exercise creation and maintenance in Artemis. These requirements address the precision bottleneck that Section 2.6 establishes, and the needs that stakeholder analysis surfaces: reliable detection, post-detection filtering, accurate localization, structured reporting, and support for multiple programming languages. Requirements FR13–FR15 address context representation enhancements that remain open goals for future work [BD09].



**FR8 Improve Inconsistency Detection Precision:** The system shall detect structural and semantic inconsistencies across exercise artifacts — including method signature mismatches, attribute type mismatches, visibility mismatches, and identifier naming conflicts — with higher precision and fewer false positives than the baseline.

**FR9 Verify Detected Issues:** The system shall re-evaluate each detected issue against the exercise artifacts before presenting the report to the instructor, filtering issues that do not correspond to real inconsistencies.

**FR10 Localize Issues Accurately:** The system shall provide file paths and line ranges for each detected inconsistency with localization quality at or above the baseline Span  $F_1$  and IoU values.

**FR11 Classify and Report Issues:** The system shall assign each detected issue a category from the consistency ontology and a severity level, and produce a structured report containing all detected issues together with execution timing and resource usage metrics.

**FR12 Support Multiple Programming Languages in Detection:** The system shall analyze exercises written in different programming languages and apply language-appropriate context filtering without requiring manual configuration from the instructor.

**FR13 Render Exercise Context:** The system shall transform heterogeneous exercise artifacts into a unified representation that preserves file structure, line numbering, and artifact boundaries.

**FR14 Represent Exercise Structure:** The system shall construct a structured representation of exercise artifacts that captures the relationships between code elements, method signatures, and problem statement requirements.

**FR15 Provide Structured Context to Analysis:** The system shall supply the structured exercise representation to the analysis process as an alternative or supplement to the plain-text context rendering, enabling the analysis to reason about explicit artifact relationships.

## 4.4 Quality Attributes and Constraints

The quality attributes describe how the system fulfills its functional requirements and follow the Usability, Reliability, Performance, and Supportability (URPS) categories defined by Bruegge and Dutoit [BD09].

**QA1 Actionable Issue Reports (Usability):** Each reported consistency issue shall contain a descriptive explanation, a category, a severity level, and precise artifact locations with file paths and line ranges. Instructors shall be able to act on each finding without additional investigation beyond reading the report.

**QA2 Graceful Degradation (Reliability):** The system shall handle missing or inaccessible repositories by degrading to partial analysis rather than aborting the entire consistency check. A failure in one analysis process shall not prevent the other analysis processes from completing and returning results.

**QA3 Practical Analysis Duration (Performance):** The system shall complete the analysis of a single exercise within a time bound that remains practical for instructor workflows. The benchmarking workflow shall support concurrent execution of multiple variant evaluations to keep total evaluation time manageable for datasets with hundreds of variants.

**QA4 Extensible Analysis Pipeline (Supportability):** The analysis pipeline shall support the addition of new analysis processes, new consistency issue categories, and new programming languages without requiring modifications to existing components. The benchmarking framework shall accept new detection configurations without changes to the evaluation and reporting logic.

Technical, organizational, and external factors impose boundary conditions under which the system must operate. These conditions influence design decisions without describing functional behavior directly [BD09].

**C1 Platform Integration:** The system operates within an existing learning management platform and must conform to its architecture, module structure, and role-based access control model. All extensions must integrate into the existing codebase without disrupting other platform features.

**C2 External AI Provider Dependency:** The analysis relies on external AI model providers that the system reaches via network APIs. These providers introduce latency, availability constraints, and potential changes to model behavior across versions. The system must tolerate provider-side failures without data loss.

**C3 Cost Efficiency:** Each analysis invocation consumes paid API resources. The per-run cost must remain practical for educational institution budgets, particularly when the benchmarking workflow executes hundreds of variant evaluations in a single run.

**C4 Reproducibility:** A versioned dataset, deterministic evaluation scripts, and documented model configurations must enable any developer to reproduce all evaluation results. Any developer who executes the same benchmark run must obtain comparable results.

**C5 Data Confidentiality:** Exercise artifacts that university instructors contribute may contain materials that courses actively use. Test suites and solution repositories must not become publicly accessible. The dataset structure must support full evaluation without exposing confidential materials to unauthorized users.

## 4.5 System Models

This section illustrates the requirements through three complementary system models [BD09]. The scenarios describe concrete usage situations from the perspective of the two primary actors — instructors and developers — and ground the functional requirements in realistic workflows. The domain model defines the core domain concepts and their relationships independently of any implementation. The dynamic model captures the behavioral flow of the benchmarking workflow from the developer’s perspective — from dataset preparation and variant upload through consistency check execution to result publication.

### 4.5.1 Scenarios

**Visionary Scenario — Zero False Positive Consistency Check**

Professor Weber, an instructor for Introduction to Programming, finishes editing a multi-file Java exercise and triggers the consistency check. The system analyzes the problem statement, template repository, solution repository and tests repository within 30 seconds and returns two detected issues: a *method return-type mismatch* between the problem statement and the template, and an *identifier naming inconsistency* between the problem statement and the solution. Both issues include precise file paths, line ranges, and a description that explains the contradiction. Professor Weber navigates to each location, confirms that both findings are genuine errors, and corrects the exercise before students access it. The report contains no false positives, so the instructor does not spend time dismissing incorrect findings. This scenario illustrates the precision goal that motivates FR8, FR9, and FR11 — a report that contains only genuine issues and requires no manual dismissal.

### **Demo Scenario — Benchmarking a New Detection Approach**

A developer implements a verification checker that re-evaluates detected issues to filter false positives. The developer triggers the benchmarking workflow on the extended dataset, which contains exercise variants across multiple programming languages. The system creates a test course on the running platform instance, uploads all exercise variants, executes the consistency check on each variant using the modified detection pipeline, and compares the results against the gold-standard annotations. The evaluation report shows that the new approach improved the  $F_1$  score and reduced the number of false positives compared to the baseline that the developer defines, while the per-exercise breakdown reveals which exercise types and programming languages benefit most from the verification step. The developer compares this report against the baseline report that the workflow produced under identical conditions to confirm that the improvement is consistent across the dataset. This scenario illustrates the benchmarking workflow that motivates FR1 through FR4 — reproducible evaluation of any detection approach under identical conditions.

### 4.5.2 Domain Model

This section presents the domain model of the consistency check system as a UML class diagram, following the structure that Bruegge and Dutoit propose [BD09]. Figure 6 presents the model. The model covers the exercise material that instructors create, the issues that the detection pipeline produces, and the benchmarking component that developers use to evaluate detection approaches against the produced reports.

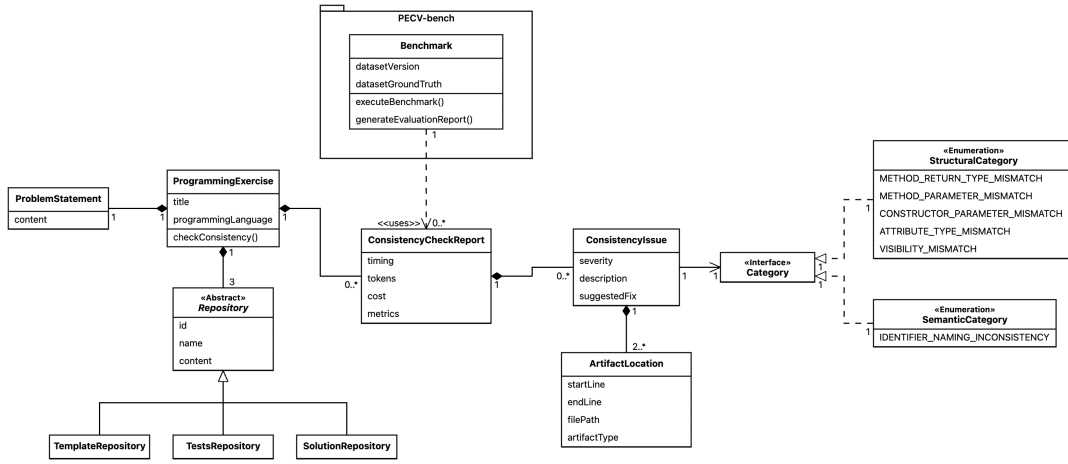


Figure 6: Domain model of the consistency check system in Artemis. The *Benchmark* component in PECV-bench uses the *ConsistencyCheckReport* through a dependency relationship, reading the JSON output to evaluate detection approaches against ground-truth annotations.

*ProgrammingExercise* forms the central domain object of the model. It stores the attributes `title`, `programmingLanguage`, and `problemStatement`, and composes three *Repository* objects that represent the template, the solution, and the test artifacts of an exercise. Each *Repository* carries the attributes `id`, `name`, and `content`. *ProblemStatement* composes with *ProgrammingExercise* through a composition relationship with cardinality 1, reflecting that every exercise owns exactly one problem statement and that the problem statement does not exist independently of the exercise that owns it. Through the operation `checkConsistency()`, *ProgrammingExercise* captures the QA activity that the Hyperion pipeline performs and that produces a *ConsistencyCheckReport* (FR8, FR9, FR11).

*ConsistencyCheckReport* aggregates zero or more *ConsistencyIssue* objects for a single exercise and records execution metadata that developers inspect through the benchmarking workflow: the timing, tokens, and cost attribute groups cover start and end times with duration, prompt, completion, and total token counts, and prompt, completion, and total cost values (FR1, FR11). Instructors see the issue list through the Artemis user interface, whereas developers see the full report including the metadata groups through the JSON response that the consistency check endpoint returns.

*ConsistencyIssue* stores the attributes severity, description, and suggestedFix, and references exactly one *Category* through specification inheritance (FR8, FR11). *Category* is modeled as an interface that *StructuralCategory* and *SemanticCategory* implement. *StructuralCategory* enumerates the five structural subcategories of the ontology and *SemanticCategory* enumerates the identifier naming subcategory. The two subtypes reflect the ontology that Section 2.2 defines and align the model with the detection prompts that target each subcategory. Each *ConsistencyIssue* composes two or more *ArtifactLocation* objects through a minimum cardinality that enforces the cross-artifact nature of an issue: a report entry must reference at least two locations across different artifacts to constitute a genuine contradiction (FR10). *ArtifactLocation* stores the attributes filePath, startLine, endLine, and artifactType. The artifactType attribute identifies the affected artifact as the problem statement, template, solution, or test repository.

The *Benchmark* class represents the PECV-bench component that evaluates detection approaches against the ground-truth annotations stored in its dataset. It carries the attribute datasetVersion and the operations executeBenchmark() and generateEvaluationReport(), and uses *ConsistencyCheckReport* through a dependency relationship: PECV-bench reads the JSON report that the consistency check endpoint returns, compares the reported issues against its internal gold-standard annotations, and aggregates detection metrics across all variants (FR1, FR3, FR4). Dietrich et al. [Die+25] describe the internal structure of PECV-bench<sup>6</sup>

---

<sup>6</sup><https://github.com/ls1intum/PECV-bench>

in detail, including dataset organization, variant materialization, and evaluation scripts. The dependency arrow expresses that the two systems share the JSON data without implying direct object references between them.

### 4.5.3 Dynamic Model

The activity diagram models the dynamic behavior of the consistency check benchmarking workflow, showing how a developer evaluates a detection approach. The model uses UML activity diagram notation as recommended by Bruegge and Dutoit [BD09].

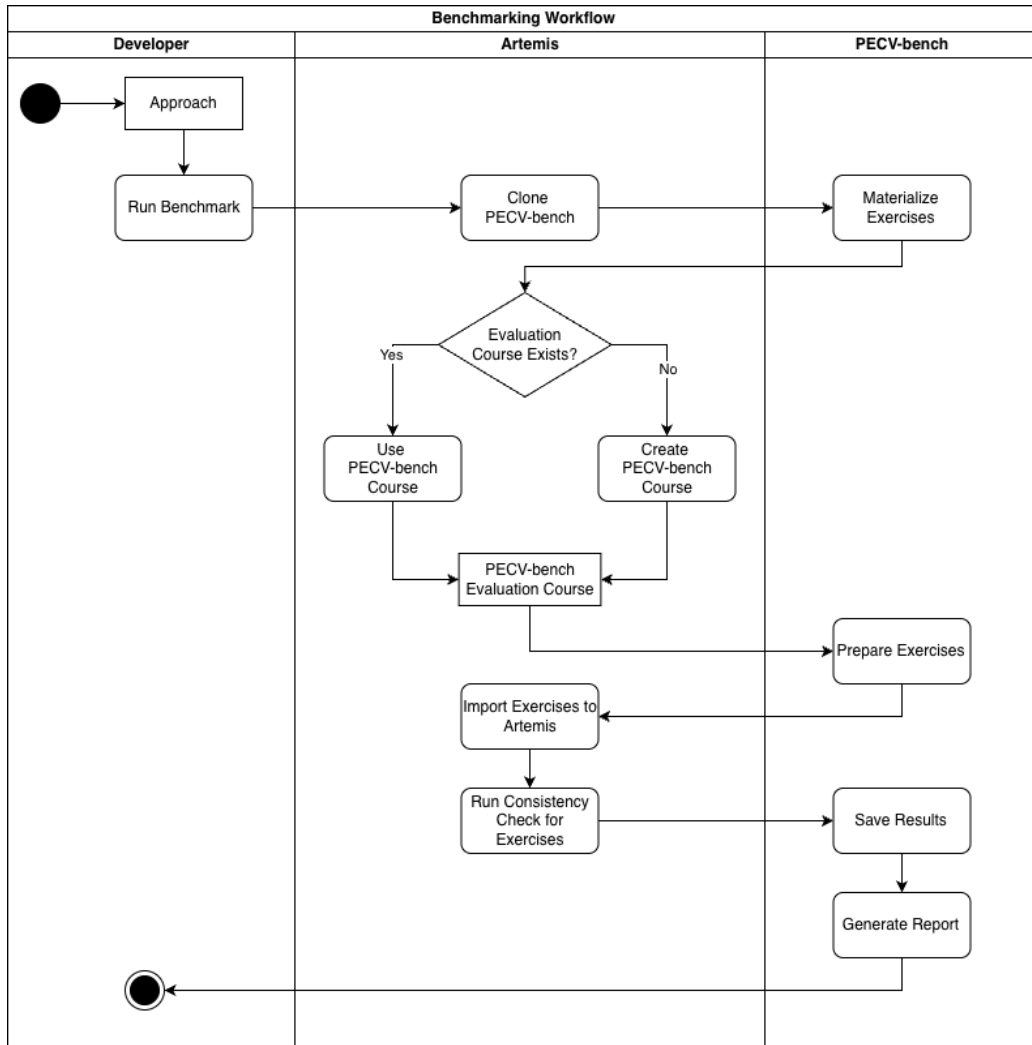


Figure 7: Activity diagram of the benchmarking workflow across three actors: the developer, the Artemis platform, and PECV-bench. The developer selects a detection approach and triggers the benchmark run.

The workflow (Figure 7) begins when a developer initiates an evaluation run against a running Artemis instance. Artemis clones the PECV-bench repository and delegates variant materialization and exercise preparation to PECV-bench (FR5, FR6). The course preparation phase checks whether an evaluation course already exists on the Artemis instance and creates one if not; both paths converge on a ready evaluation course before exercise import begins. PECV-bench prepares the exercise variants and Artemis imports them into the evaluation course, connecting the dataset to the same consistency check pipeline that instructors use in production (FR2).

The evaluation phase executes the consistency check on each uploaded variant and collects the structured issue reports that the pipeline produces (FR3). The reporting phase compares the detected issues against the gold-standard annotations, computes detection and localization metrics, and produces per-variant and per-model performance breakdowns (FR1, FR4). The workflow concludes when the benchmarking scripts generate the results of the evaluated pipeline configuration, enabling direct comparison against the baseline and against other detection approaches under identical conditions (C4).

## 5 Architecture

This chapter follows the System Design Document Template by Bruegge and Dutoit [BD09] and maps the application domain concepts from Chapter 4 to the solution domain. Where Chapter 4 describes the required system behavior independently of technology, this chapter describes how the solution domain realizes these requirements. The chapter covers the architectural overview, design goals and their trade-offs, the subsystem decomposition with a component diagram, the dataset structure, the persistence strategy, and the access control model. Chapter 6 and Chapter 7 cover the step-by-step implementation of the benchmarking workflow and the dataset construction process that this architecture supports.



## 5.1 Overview

The architecture centers on the Artemis platform, which hosts the Hyperion consistency check subsystem. The Hyperion service receives exercise data from the Artemis core, renders exercise artifacts into a unified context, invokes external LLM provider APIs to perform consistency checks, and returns a structured report to the instructor. The benchmarking workflow runs within the Artemis development environment and connects PECV-bench to the Artemis consistency check through the Artemis REST API.

## 5.2 Design Goals

The design goals derive from the quality attributes and constraints that Chapter 4 defines and guide the architectural decisions throughout the system. The following text prioritizes these goals; where goals conflict, the text states the rationale for each trade-off explicitly [BD09].

Detection precision receives the highest priority among all design goals. Every architectural decision favors reducing the number of false positives that reach the instructor, even at the cost of additional computation. The verification checker (FR9) adds a sequential analysis step after the parallel structural and semantic checkers. This step introduces one additional LLM invocation per consistency check, which increases both latency and API cost. The finding from Section 4.2 that nearly one in two reported issues at baseline does not correspond to a real inconsistency justifies this trade-off — eliminating even a fraction of these false alarms delivers more practical value than faster execution.

Extensibility ranks second. The analysis pipeline must accommodate new programming languages, new analysis processes, and new context representations without requiring modifications to existing components (QA4). The language filter service achieves this through a strategy pattern: each programming language registers its own exclusion rules, and languages without a registered strategy fall back to a global default. New checkers plug into the parallel execution stage alongside the existing structural and semantic checkers. This registration-

based architecture adds indirection compared to hardcoded logic, but it enables independent development and testing of each component.

Traceability and reproducibility rank third. The benchmarking workflow must enable any developer to reproduce a specific evaluation run and verify its results (C4). Each run records an approach identifier derived from the git branch and commit hash of the Artemis codebase, the model configuration and reasoning effort, execution metadata including timestamps and case counts, and a code snapshot of the current Hyperion codebase and prompt templates. These artifacts enable any developer to recover the exact system state at the time of evaluation. The trade-off is storage overhead: each run produces metadata files, result JSON files, and a source archive.

Cost efficiency ranks fourth. Each consistency check invocation consumes paid LLM API tokens, and the benchmarking workflow executes hundreds of checks per evaluation run (C3). The system reduces token usage by filtering irrelevant files — build artifacts, binaries, IDE configurations, and oversized files — before context rendering. A size guard excludes files exceeding 100 kilobytes. The trade-off is that aggressive filtering risks excluding files that contain relevant information; the exclusion patterns must balance completeness against token cost without silently dropping important context.

### 5.3 Subsystem Decomposition

The system decomposes into three subsystems that together realize the benchmarking workflow and the production consistency check: the Artemis Server, which hosts the Hyperion consistency check module and the Evaluation Workflow, PECV-bench, which manages the dataset and evaluation logic, and PECV-bench-dataset<sup>7</sup>, which stores confidential exercise components. An external LLM provider supplies the reasoning backend through a network API. Figure 8 shows how these subsystems interact through their provided and required interfaces.

---

<sup>7</sup><https://github.com/ls1intum/PECV-bench-dataset>

### 5.3.1 Hyperion Consistency Check Module

The Hyperion module realizes the consistency check pipeline as a set of collaborating components within the Artemis Server. Three components share responsibility for the analysis: Context Filtering selects which files enter the pipeline, Context Rendering assembles the filtered files into a unified text representation, and Consistency Check drives the actual detection across the rendered context. The module exposes its capability through the Benchmark interface that the Evaluation Workflow consumes and through the LLM-Service interface that connects it to the external LLM provider.

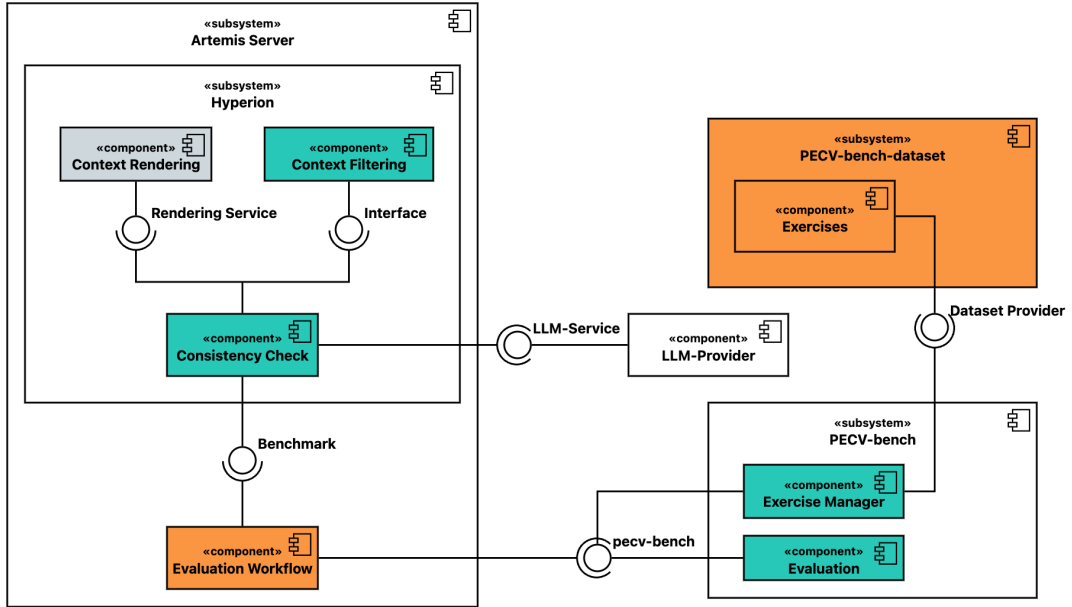


Figure 8: Component diagram of the system architecture. Colours indicate the contribution status of each component: ■ new in this thesis, ■ extended in this thesis, ■ reused and identified as open for future work.

The Context Filtering component selects which files from each repository enter the analysis (FR12). It maintains a registry of language-specific exclusion strategies. Each strategy combines global exclusion patterns for version control metadata, IDE configurations, build artifacts, and binary files with language-specific patterns — for example, build wrappers for Java or virtual environments for Python. A

safety net of allowed file extensions prevents unrecognized binary files from entering the context, and a size guard excludes files that exceed a configured threshold. Languages without a registered strategy fall back to global-only filtering. The strategy registry realizes the extensibility design goal (QA4): new languages plug in without changes to the filtering logic.

The Context Rendering component transforms the filtered files into a unified text representation that preserves line numbers, directory structure, and artifact boundaries (FR13). The thesis reuses this component from the existing Hyperion implementation without modification and identifies structured context representations as a future extension to this component, which the grey shading in Figure 8 indicates.

The Consistency Check component orchestrates the detection and exposes the pipeline to callers. It combines a structural checker and a semantic checker, both of which consume the rendered context and invoke the LLM provider through the LLM-Service interface to produce typed consistency issues bounded by the ontology (FR8). The component extends this two-checker design with the verification checker that this thesis introduces, which re-evaluates each reported issue to filter false positives before the report reaches the instructor (FR9). The component tolerates checker-level failures by returning an empty issue list for the failed checker rather than aborting the entire analysis, which preserves partial results under provider-side errors (QA2).

### **5.3.2 Evaluation Workflow**

The Evaluation Workflow component resides inside the Artemis Server and represents the bridge between the Hyperion module and PECV-bench. It provides an internal Benchmark interface that triggers a benchmark run and requires the pecv-bench interface for variant materialization and report generation (FR2, FR3). The component keeps benchmark orchestration separate from the consistency check pipeline itself: Hyperion remains a pure detection service, and the Evaluation Workflow takes on the responsibility for coordinating dataset preparation, exercise

upload, consistency check invocation, result collection, and report generation across the two subsystems (FR1, FR4).

The workflow takes its runtime parameters — server configuration, dataset version, model configuration, and exercise selection — from an external configuration source rather than hardcoded values, which decouples workflow execution from the build and supports reproducibility across evaluation runs (C4). The component records an approach identifier derived from the state of the Artemis codebase and a code snapshot of the Hyperion sources active during the run, which together preserve the exact system state behind every reported result.

### 5.3.3 PECV-bench and PECV-bench-dataset

PECV-bench provides the dataset management and evaluation logic that the Evaluation Workflow consumes through the pecv-bench interface. The Exercise Manager component encapsulates the logic for materializing variants from base exercises and patch files that inject individual inconsistencies (FR5). The Evaluation component encapsulates the alignment between detected issues and gold-standard annotations and produces the detection and localization metrics — precision, recall,  $F_1$ , Span  $F_1$ , and IoU — together with the per-variant and per-model reports that the benchmarking workflow consumes (FR1, FR3, FR4). Both components extend prior implementations to cover the multi-language dataset that this thesis introduces.

The PECV-bench-dataset<sup>8</sup> subsystem exposes confidential exercise components — test suites and solution repositories for exercises that university instructors actively use in their courses — through the Dataset Provider interface (FR7, C5). The separation of public and confidential components into two subsystems realizes the data confidentiality constraint at the architecture level: PECV-bench can remain publicly accessible for reproducibility purposes while PECV-bench-dataset enforces authorization on the materials that must not become public.

The external LLM provider serves as the reasoning backend that the Consistency Check component invokes through the LLM-Service interface. The system

---

<sup>8</sup><https://github.com/lslintum/PECV-bench-dataset>

depends on a single active provider per run, which keeps the configuration simple and matches the reproducibility design goal (C2, C4).

## 5.4 Dataset Structure and Extension

The baseline dataset contains three Java exercises from a single course with 91 perturbed variants and 93 annotated consistency issues (FR5, FR6). Each exercise contains a clean base version and a variants directory. Each variant holds a patch file that injects one inconsistency relative to the base, a gold-standard annotation that defines the expected issues, and a category description. The Exercise Manager materializes a variant by copying the base artifacts and applying the corresponding patch.

This thesis extends the dataset with a second version (V2) that draws exercises from six university courses taught by different instructors at TUM. V2 covers programming languages beyond Java, including Python, C, Assembler, SQL, and Swift. The versioned directory layout separates the original and extended datasets, with each version containing course directories and exercise directories beneath them. Each V2 exercise follows the same structure as V1 and adds an exercise-details file that specifies the programming language so that the language-aware filter applies the correct strategy during analysis (FR6, FR12). Chapter 7 describes the collection, injection, and annotation protocol that produces the V2 variants.

The dataset design satisfies FR7 and C5: the public components — problem statements and template repositories — reside in the PECV-bench subsystem, while confidential components — test suites and solution repositories — reside in the separate PECV-bench-dataset subsystem. The Evaluation Workflow merges both subsystems before materialization, which enables full evaluation while protecting materials that courses actively use.

## 5.5 Persistent Data Management

The Artemis platform stores the benchmarking course and all imported exercise variants in its PostgreSQL database during evaluation. The course manager script can delete and recreate this course between evaluation runs, ensuring a clean state

for each benchmark execution. The REST endpoint returns the consistency check response — containing the detected issues, timing, token usage, and cost metadata — directly to the caller.

The PECV-bench subsystem serves as the persistent store for all evaluation artifacts. The Evaluation Workflow organizes result files hierarchically by approach identifier, dataset version, run identifier, and exercise variant, and stores each consistency check response as a separate file under that hierarchy. A parallel metadata hierarchy captures the model configuration, reasoning effort, codebase state, timestamp, and execution statistics of each run. The code snapshot archive that the workflow attaches to every run preserves the Hyperion sources and prompt templates active during that run (C4). Generated reports — aggregate summaries, per-variant analyses, and scatter plots — reside alongside the run directories. This organization produces a permanent and versioned record of every evaluation that any developer can reproduce and compare against prior runs.

## 5.6 Access Control

The Artemis platform restricts access to the consistency check functionality through role-based authorization. Only users with editor privileges or above can trigger a consistency check for a specific exercise, and only users with course-level editor access can modify problem statements. The benchmarking workflow authenticates as an administrator during evaluation runs, which grants full access to course creation, exercise import, and consistency check execution. Repository-level permissions protect confidential exercise components in the private dataset repository and restrict visibility to authorized developers (C5).

## 6 Benchmarking Workflow

This chapter describes the implementation of the first objective: the on-demand benchmarking workflow that connects the PECV-bench evaluation framework to the Artemis production system. It covers the end-to-end pipeline from environment setup to result publication and explains the key design and implementation decisions. Figure 9 shows the activity diagram of the workflow and structures this

chapter — every subsequent section corresponds to one group of activities in the diagram.

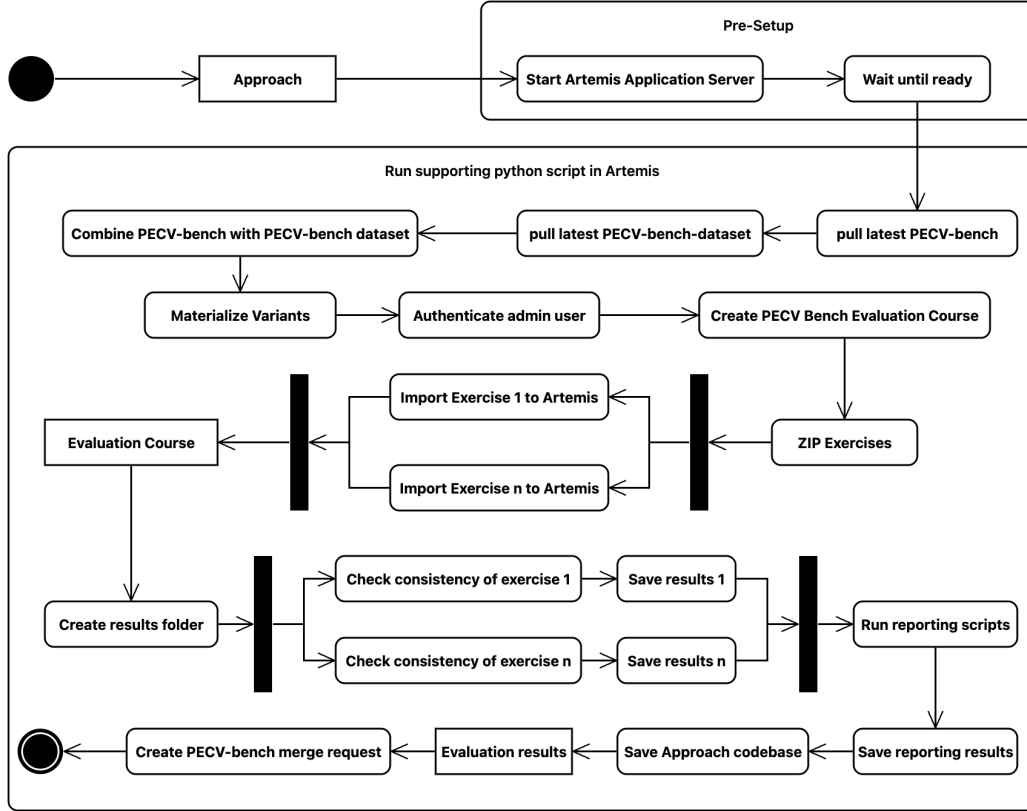


Figure 9: Activity diagram of the benchmarking workflow. The workflow starts after the developer boots a local Artemis application server and waits for it to become ready.

## 6.1 Workflow Overview

The workflow automates the evaluation of a consistency check approach against the annotated dataset by executing 14 sequential steps through a single entry point, `run_pecv_bench.py`. A central configuration file `config.ini` provides all runtime parameters: the Artemis server URL, administrator credentials, dataset version, exercise selection, model name and reasoning effort, and the code snapshot manifest. The workflow reads the file once at startup and propagates the values to every subsequent module, which keeps runtime behaviour fully configurable without code changes. All benchmarking scripts reside inside the Artemis repos-



itory under `supporting_scripts/hyperion/consistency-check-benchmark`<sup>9</sup>, which keeps the workflow versioned alongside the Hyperion module it evaluates.

The entry point delegates each phase to a dedicated Python module — `exercises.py`, `course.py`, `consistency_check.py`, `report.py`, `code_snapshot.py`, and `merge_request.py`. Each module exposes the same step-numbered interface that the entry point uses, which means the developer can also run any module in isolation to recover from a failure at a specific step without re-executing the earlier phases.

The workflow produces three outputs that together realize the reproducibility design goal (C4). The per-variant JSON responses from the consistency check endpoint land in the PECV-bench results hierarchy under `results/{approach_id}/{version}/{run_id}/cases/`. A code snapshot ZIP captures the Hyperion source files and prompt templates active during the run. A pull request in the PECV-bench repository carries the generated report with plots and a reference comparison row for the recorded approach. Chapter 7 describes the dataset structure and the two dataset versions that this chapter refers to throughout.

## 6.2 Environment Setup and Configuration

The workflow expects three prerequisites before execution. The developer creates a Python 3.13 virtual environment inside the `consistency-check-benchmark` directory and installs the pinned dependencies from `requirements.txt`. The developer installs and authenticates the GitHub Command-Line Interface (`gh`), which the final step of the pipeline uses to create the pull request programmatically. A local Artemis instance must run and expose the URLs that `config.ini` defines.

The entry point reads every runtime parameter from `config.ini` at startup. The `[Settings]` section provides administrator credentials and server URLs. The `[PECVCourseSettings]` section controls whether the workflow creates a new benchmark course or reuses an existing one, and specifies the PECV-bench and PECV-bench-dataset repository URLs and the target branch. The

---

<sup>9</sup>[https://github.com/lslintum/Artemis/tree/develop/supporting\\_scripts/hyperion/consistency-check-benchmark](https://github.com/lslintum/Artemis/tree/develop/supporting_scripts/hyperion/consistency-check-benchmark)

[`PECVExerciseSettings`] section defines the dataset version and the set of courses and exercises to include. The [`PECVConsistencyCheckSettings`] section specifies the model name and reasoning effort used to label results, the subset of exercises to include in the consistency check run, the file list for the code snapshot, and the reference row used for comparison in the generated report.

Steps 1 through 3 initialize the evaluation environment. Step 1 resolves the local path of the PECV-bench working directory. Step 2 clones or pulls both the public PECV-bench repository and the confidential PECV-bench-dataset repository, then merges the `data/` subtree of the dataset repository into the PECV-bench working directory while preserving the identical path structure. This merge step realizes constraint C5 at runtime: confidential test suites and solution repositories remain in a separate private repository that only authorized developers can clone, yet the merged working directory behaves as a single dataset for every subsequent step. Step 3 installs the PECV-bench Python package in editable mode, which exposes the PECV-bench CLI and its `VariantManager` to the subsequent steps.

### 6.3 Variant Materialization and Course Preparation

The second phase combines two independent activities from Figure 9: variant materialization on the PECV-bench side and course preparation on the Artemis side.

Step 4 materializes all variants for the configured dataset version. The workflow imports `VariantManager` from the PECV-bench CLI and calls `materialize_variant()` for each variant. Materialization copies the base exercise artifacts — `problem-statement.md`, `solution/`, `template/`, `tests/`, and `exercise-details.json` — into the variant directory and applies the variant-specific git patch. The patch encodes the exact inconsistency that the variant injects into the base exercise.

Steps 7 and 8 create a fresh benchmark course on the Artemis server. The workflow first checks whether a course with the configured short name already exists and deletes it through retry logic before creating a new one. The `create_course`

parameter in `config.ini` allows the developer to skip course creation and reuse an existing benchmark course instead, which shortens iteration time during local development.

Step 9 converts each materialized variant into the ZIP format that the Artemis import endpoint expects. For every variant, the workflow creates intermediate ZIP archives of the `solution/`, `template/`, and `tests/` directories and packages them together with the `exercise-details.json` file — which the step updates with the server course identifier, a prefixed title of the form `{variant_id} - {exercise_name}`, and a sanitized short name — into a final archive named `{variant_id}-FullExercise.zip`. The workflow renames the template archive to `{variant_id}-exercise.zip` so that it matches the Artemis import convention.

Step 10 imports every variant ZIP file into the benchmark course through `POST /api/programming/courses/{courseId}/programming-exercises/import-from-file`. The imports execute concurrently through a thread pool whose size the `max_threads` parameter in `config.ini` controls. The workflow logs failed imports without aborting the pipeline; it reports them at the end of the phase, and the developer reruns the import step in isolation without repeating the earlier phases.

## 6.4 Consistency Check Execution

Step 11 retrieves the Artemis server identifiers of all imported exercises through `GET /api/programming/courses/{courseId}/programming-exercises`. The workflow parses the returned exercise titles — which the import step formats as `{variant_id} - {short_code} - {description}` — back into the local `{short_code}-{description}:{variant_id}` format that the dataset uses. The result is a mapping from local variant identifiers to server exercise identifiers that the consistency check runner consumes in the next step.

Step 12 triggers the consistency check for every variant through `POST /api/hyperion/programming-exercises/{exerciseId}/consistency-check` with the `skipThreadContext=true` parameter, which forces the endpoint to run synchro-

nously in the request thread rather than dispatching to an asynchronous worker. Checks execute concurrently through the same thread pool that the import step uses. The workflow derives the approach identifier from the current git branch and short commit hash of the Artemis repository, following the pattern `artemis-{branch}-{commit}`. The workflow builds the run identifier from the configured model name, a Coordinated Universal Time (UTC) timestamp, and the same short commit hash, which uniquely distinguishes multiple runs of the same approach.

The workflow stores each response as a separate JSON file under `pecv-bench/results/{approach_id}/{version}/{run_id}/cases/{course}/{exercise}/{variant}.json` and injects `case_id` and `run_id` fields into the payload for later aggregation. A parallel YAML metadata file under `runs/{approach_id}/{version}/{run_id}.yaml` records the approach identifier, model arguments, git branch and commit, and the count of successful and failed cases for the run.

## 6.5 Evaluation and Reporting

The fourth phase generates the evaluation reports, captures the code snapshot, and publishes the results. Figure 9 groups these activities into `Run reporting scripts`, `Save Approach codebase`, `Evaluation results`, and `Create PECV-bench merge request`.

Step 13 delegates metric computation and plot generation to the PECV-bench CLI. The workflow calls `pecv-bench variants-analysis --clear`, then `pecv-bench variants-analysis --plot`, and finally `pecv-bench report`. The `variants-analysis` command loads every JSON response from the results hierarchy, aligns each prediction with the gold-standard annotation of the corresponding variant, and computes per-variant precision, recall,  $F_1$ , Span  $F_1$ , IoU, latency, and cost. It produces a `variants_report.json` grouped by model and generates two scatter plots: one that plots input tokens against the  $F_1$  score per model across all exercises, and one that presents the same breakdown in a grid grouped by model and exercise.

The `report` command aggregates every run of the current approach into `summary.md`, `summary.json`, and `summary.tex`. The workflow then injects a reference row from `config.ini` into the summary table immediately after the approach row. This injection ensures that every generated report shows a direct numerical comparison against the published PECV-bench baseline for the corresponding dataset version.

Step 13b captures a reproducibility snapshot by creating a ZIP archive of the Hyperion source files and prompt templates active during the run. The `code_snapshot_files` key in `config.ini` declares, as a JSON map of subdirectory names to file lists, which files the workflow resolves against the Hyperion Java source directory. The workflow also includes every prompt template from `src/main/resources/prompts/hyperion/` whose filename starts with `consistency_`. The resulting archive lands at `pecv-bench/results/{approach_id}/{approach_id}.zip` next to the approach results directory.

Step 14 publishes the results to the PECV-bench repository through a pull request. The workflow creates a branch named after the approach identifier, commits the `results/` and `runs/` directories for that approach, and pushes the branch. When a remote branch with the same name already exists — for example, when a second dataset version joins an existing approach — the workflow fetches the remote and rebases onto it so that results from both dataset versions coexist in a single pull request. The workflow assembles the pull request body from the `report.md` files of every evaluated dataset version and rewrites relative image paths to absolute raw GitHub URLs so that the scatter plots render directly in the pull request description. The pull request targets the branch that the `pecv_bench_branch` parameter in `config.ini` designates. The workflow uses `gh pr create` to create a new pull request or `gh pr edit` to update the body of an existing one.

## 6.6 Limitations

Several preconditions and failure modes constrain the workflow. It requires a local Artemis instance with the Hyperion module enabled and reachable at the

configured server URL; without this prerequisite, the course creation and consistency check steps cannot complete. The approach identifier reflects the current branch and commit of the Artemis repository, so uncommitted local changes to the detection pipeline fall outside the identifier and may produce results that no longer reproduce from the recorded commit alone. Token and cost values in the generated report depend on LangSmith tracing; when the LangSmith API key is absent from the Artemis environment, those columns remain zero and the cost comparison against the reference baseline loses meaning. Both the `max_threads` parameter and the server’s capacity to handle parallel LLM calls bound the concurrent execution of consistency checks. When the server rejects requests due to rate limiting or provider-side errors, the affected variants fail individually without aborting the pipeline, which leaves the evaluation data incomplete for those variants and forces the developer to rerun the consistency check step in isolation.

## 7 Data Creation

This chapter describes the implementation of the second objective: the extension of the PECV-bench dataset from three Java exercises to a multi-language dataset covering six university courses. Figure 10 illustrates the end-to-end construction process that this chapter describes. The chapter proceeds from the overall shape of the dataset, through the data collection that surfaces candidate exercises, to the variant creation and annotation protocol that produces the final annotated variants, and closes with quantitative statistics and a discussion of limitations.

### 7.1 Dataset Overview

The extended dataset, referred to as V2, contains 16 programming exercises drawn from six courses at TUM. These include the three Java exercises that form the original V1 dataset with 91 variants (Table 3) — all from the Introduction to Programming course (ITP2425) — and 13 additional exercises collected across five further courses: Introduction to Computer Architecture (ERA2021), Introduction to Software Engineering (ISE22), Quantum Computing Lab (QCSL25), iPraktikum (IOS26), and the test and exercise collection course MTG26. The 16

exercises span six programming languages: Java, Assembler, C, Python, SQL, and Swift. The new dataset contains 325 annotated variants in total (Table 4), each representing one exercise instance with one or more injected inconsistencies and a corresponding gold-standard annotation.

| Version | Course  | Exercise                    | Variants  |
|---------|---------|-----------------------------|-----------|
| V1      | ITP2425 | H01E01-Lectures             | 30        |
| V1      | ITP2425 | H02E02-Panic_at_Seal_Saloon | 29        |
| V1      | ITP2425 | H05E01-Space_Seal_Farm      | 32        |
|         |         | <b>Total</b>                | <b>91</b> |

Table 3: Per-exercise variant counts in the V1 dataset, which contains the three Java exercises from the original PECV-bench benchmark published by Dietrich et al. [Die+25].

| Version | Course  | Exercise                                   | Variants   |
|---------|---------|--------------------------------------------|------------|
| V2      | ERA2021 | H00-Hello_World_ASM                        | 18         |
| V2      | ERA2021 | H03-Grafikspeicher                         | 18         |
| V2      | ERA2021 | P01-Raycasting_mit_Festkommazahlen         | 18         |
| V2      | IOS26   | TC1-Bookstore                              | 18         |
| V2      | IOS26   | TC2-Developer                              | 18         |
| V2      | ISE22   | H05E01-REST_Architectural_Style            | 18         |
| V2      | ISE22   | H10E01-Containers                          | 18         |
| V2      | ITP2425 | H01E01-Lectures                            | 30         |
| V2      | ITP2425 | H02E02-Panic_at_Seal_Saloon                | 29         |
| V2      | ITP2425 | H05E01-Space_Seal_Farm                     | 32         |
| V2      | ITP2425 | SE01E01-UML                                | 18         |
| V2      | MTG26   | TA1-SQL_Advanced                           | 18         |
| V2      | MTG26   | TA2-SQL_Basics                             | 18         |
| V2      | QCSL25  | QC01-Decision_Diagrams_and_Tensor_Networks | 18         |
| V2      | QCSL25  | QC02-Far_Term_Quantum_Algorithms           | 18         |
| V2      | QCSL25  | QC03-Magic_State_Distillation              | 18         |
|         |         | <b>Total</b>                               | <b>325</b> |

Table 4: Per-exercise variant counts in the V2 dataset. The three ITP2425 exercises from V1 reappear here to represent the full dataset.

Confidentiality requirements split the dataset into two repositories. Exercises whose contributing instructors permit public release reside in the public PECV-bench repository. Exercises whose instructors require confidentiality reside in the

separate private PECV-bench-dataset repository. This split allows PECV-bench to remain public for reproducibility while protecting the solution and test repository contents of restricted exercises, as Chapter 5 describes. The Quantum Computing Lab exercises currently reside in PECV-bench-dataset; PECV-bench hosts the remaining exercises directly.

## **7.2 Data Collection**

The collection phase produces the set of base exercises that the subsequent variant creation phase consumes. The section first describes the criteria that guided exercise selection, then explains how candidate exercises surfaced from the Artemis database, and finally reports which instructors contributed under which release arrangement.

### **7.2.1 Exercise Selection Criteria**

The selection criteria aimed to maximize language and structural diversity while keeping the exercises representative of the real-world exercise quality that instructors encounter in Artemis. Two conditions guided inclusion. First, each exercise had to carry sufficient pedagogical complexity to produce non-trivial variants; exercises with trivial or near-empty implementations fell outside the scope. Second, the contributing instructor had to consent to benchmark use, either publicly or under the confidentiality arrangement described above.

### **7.2.2 Sources and Contributors**

The thesis author ran a production SQL query against the Artemis database in collaboration with Artemis stakeholders to identify candidate exercises. The query retrieved the distribution of programming languages and course sizes across active exercises and surfaced the courses with the largest student populations and the broadest language coverage. The thesis author then contacted the instructors responsible for the five target courses directly — in person and by email — and asked them to contribute exercises from their course repositories.

Instructors from the Introduction to Computer Architecture, Introduction to Software Engineering, Introduction to Programming, iPraktikum, and MTG



courses agreed to share exercises for public inclusion in PECV-bench. Instructors from the Quantum Computing Lab course agreed to contribute under the confidentiality arrangement: their exercises remain in the private PECV-bench-dataset repository and do not become publicly available, but support benchmarking to improve both Artemis and the quality of their exercises. Figure 10 shows how the six courses feed into the construction process and how the resulting base exercises split between the public and the private repository.

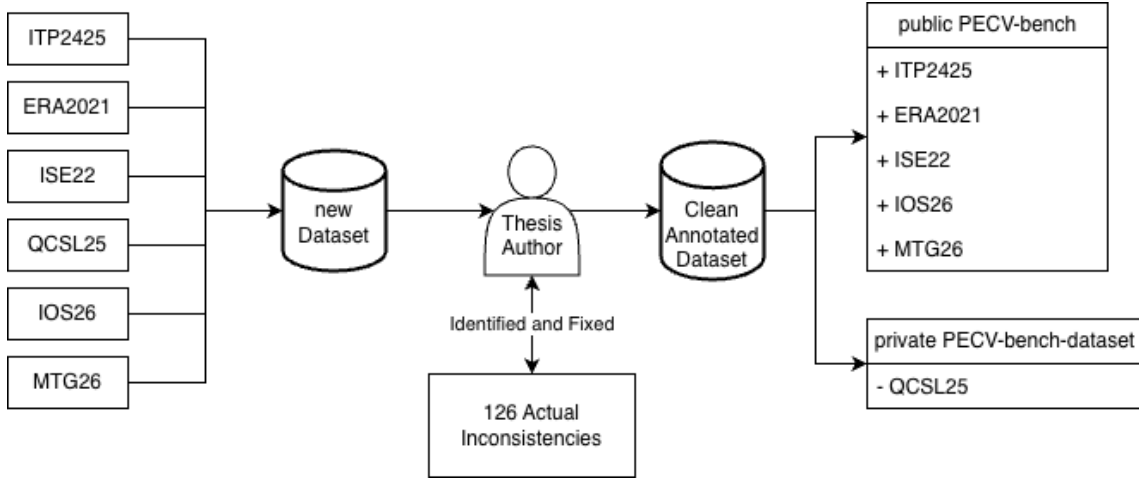


Figure 10: End-to-end construction of the V2 dataset. Six university courses contribute raw exercises that the thesis author consolidates into a single working dataset. The thesis author identifies and fixes 126 pre-existing inconsistencies to produce a clean annotated dataset, then distributes the resulting base exercises across the public PECV-bench repository and the private PECV-bench-dataset repository according to the confidentiality arrangement each instructor accepts.

### 7.3 Variant Creation and Annotation

The variant creation protocol transforms each clean base exercise into a set of annotated variants that the detection pipeline can evaluate. The protocol has three stages: identification and remediation of pre-existing inconsistencies in the raw exercises, injection of synthetic inconsistencies that form the variants, and production and manual verification of the gold-standard annotations. The following subsections describe each stage in full detail so that other researchers

can reproduce the procedure, and close with an explicit discussion of the bias controls that the design builds in.

### **7.3.1 Remediation of Pre-Existing Inconsistencies**

The thesis author received the 13 new exercises as-is from the contributing instructors and applied the baseline consistency check pipeline from Chapter 2 to every exercise in its unmodified state. Each exercise passed through the pipeline with model `azure-openai-gpt-5-mini` at `reasoning_effort=medium` — the same configuration that the baseline evaluation in Chapter 8 uses. The pipeline ran repeatedly on each exercise until four consecutive runs produced no previously unseen issues, which operationalizes the stopping rule as stability across consecutive runs rather than a fixed run count. This procedure surfaced 126 pre-existing inconsistencies across the 13 exercises — structural and semantic inconsistencies that the instructors had not noticed in their production exercises.

The thesis author manually inspected every reported issue, discarded false positives after cross-referencing the cited artifact locations, and remediated confirmed true positives in consultation with the contributing instructor when the correct resolution was ambiguous. Each base exercise thus served as a clean, internally consistent starting point for synthetic injection. This remediation step also acts as an independent second pass over the baseline output, because the thesis author confirmed or rejected every reported issue before accepting it into the remediation log.

### **7.3.2 Inconsistency Injection**

Variant creation required an injection procedure that combines scale with single-issue discipline — each variant must contain exactly one inconsistency of a known category. The thesis author developed a local LLM-based agent pipeline to support this procedure. The pipeline used model `Google Gemini 3.1 pro` at `reasoning_effort=high` and ran in two sequential stages per variant.

In the first stage, an injection agent consumed the clean base exercise, the target inconsistency category drawn from the six-category ontology defined in Chapter 2, and a system prompt that instructed the agent to produce a unified

git diff that introduces exactly one inconsistency of the target category into exactly one artifact. The target category rotation followed a fixed distribution that balances the six categories across the variants of each exercise; the thesis author fixed this distribution before generation and did not adjust it based on detector performance, which isolates variant creation from detection outcomes. The injection prompt additionally instructed the agent to avoid secondary side effects such as follow-on compilation errors or renamings that propagate beyond the targeted artifact.

In the second stage, a verification agent re-executed the baseline consistency check pipeline on the modified exercise with the same model configuration as the remediation step (Google Gemini 3.1 pro, `reasoning_effort=High`) and confirmed two conditions: the baseline detects the injected inconsistency, and the baseline reports no unintended secondary issues that the injection caused. When the verification agent reported failure on either condition, the injection stage re-ran with an adjusted prompt that referenced the specific failure. The thesis author capped the retry count at three. Variants that did not satisfy both conditions within three attempts were discarded. This procedure serialized each accepted variant as a git patch file at `variants/{variant_id}/{variant_id}.patch` in the PECV-bench dataset structure.

Supporting the multi-language dataset required extending the PECV-bench reference pipeline with a language-aware context filtering mechanism. The thesis author extended `pecv-reference/` with a `filter_files_by_language` function and per-language `LanguageConfig` objects — one each for Java, Python, Assembler, C, SQL, and Swift — that specify the file extensions, source directories, and exclusion patterns relevant to that language. This extension was a prerequisite for producing correct rendered contexts for Assembler and SQL exercises, whose file structure differs substantially from the Java exercises in V1. The language filter strategy subsequently served as the design model for the `HyperionProgrammingLanguageContextFilterService` in Artemis that Chapter 8 evaluates.

### 7.3.3 Gold-Standard Annotation and Manual Verification

The thesis author produced each gold-standard annotation by executing the baseline consistency check pipeline on the materialized variant and recording the detected issue as the expected output. The annotation file lives alongside the patch at `variants/{variant_id}/{variant_id}.json` and a human-readable category description file at `{variant_id}.{category}.md`. Each annotation records the issue category, severity, description, and the related artifact locations with file path, start line, and end line for each affected artifact. Location accuracy is critical because the evaluation metrics include Span  $F_1$  and IoU scores that measure the overlap between predicted and gold artifact spans.

Every one of the 325 variant annotations then passed through a manual verification step. The thesis author opened each annotation alongside the underlying variant and confirmed three properties: the `related_locations` entries point to the correct lines in the correct artifact files, the `category` field matches the injected inconsistency type, and no spurious issue from the base exercise carried into the annotation. Annotations with imprecise locations received corrections before inclusion in the dataset. This manual verification step ensures that the dataset provides reliable ground truth for precision and recall computation regardless of any residual bias from the detector-assisted generation procedure.

### 7.3.4 Justification and Bias Controls

The variant creation protocol uses the baseline consistency check pipeline in two roles: as a detector during remediation of pre-existing issues, and as a filter in the verification agent during injection. This design choice trades efficiency against a well-known risk — detector-assisted generation and annotation can bias a benchmark toward the evaluated system because the gold standard already contains the kinds of issues the detector can find. The alternative — manual construction of every variant and every annotation from scratch — would require an order of magnitude more effort for a dataset of this size and was not feasible within the scope of this thesis.

Four safeguards address the bias risk. The injection category distribution is fixed in advance and independent of detector performance, which removes the feedback loop between what the detector finds and what the benchmark tests. The remediation step produces an independent manual review of the baseline output rather than accepting it unchanged, so that clean base exercises do not inherit the baseline’s characteristic error patterns. The verification agent only accepts a variant when the baseline both detects the intended injection and reports no unintended side effects, which enforces single-issue discipline. Finally, the manual annotation verification step performs an end-to-end human review of every one of the 325 annotations against the underlying variant; this step does not rely on the detector’s output and therefore catches errors that the automated pipeline cannot catch. The residual risk — that all six ontology categories correspond to inconsistency types the baseline detector targets — is inherent to a benchmark that evaluates exactly this detector family rather than a weakness of the construction protocol. The threats to validity section of Chapter 8 revisits this point.

## 7.4 Limitations

The dataset covers 16 exercises drawn from six courses at a single university, which limits the generalizability of evaluation results to exercise styles and pedagogical conventions used elsewhere. The variant creation protocol relies on the baseline consistency check pipeline both to surface injection targets and to verify the resulting variants, which means that inconsistency types the baseline pipeline systematically fails to detect are underrepresented in the dataset. The bias controls described above reduce this risk but do not eliminate it entirely.

The number of exercises per language is uneven: Java has four exercises and all three Assembler exercises originate from the same course, while Swift and SQL contribute two exercises each. Evaluation results for underrepresented languages must therefore be read as directional rather than conclusive, because the per-language exercise count is insufficient to separate language-specific behavior from exercise-specific factors. The gold-standard annotations reflect single-issue variants with synthetic injections; real exercises can contain multiple simultaneous

inconsistencies of varying severity, a scenario that the current dataset design does not cover.

## 8 Consistency Check Approaches

This chapter describes the implementation and evaluation of the third objective: improving the detection precision of the Hyperion consistency check pipeline. It presents the baseline approach, the three realized improvements — the language-aware context filter, the verification checker, and the unified consistency checker — and three open approaches that this thesis identifies as future work. The chapter closes with an overall discussion of results across all approaches and a threats to validity analysis.

### 8.1 Evaluation Design

This section defines the benchmark configuration, the evaluation metrics, and the three research questions that structure the chapter. All three evaluated approaches share these settings so that results remain directly comparable.

#### 8.1.1 Benchmark Configuration

The evaluation runs all three approaches on the full V2 dataset — 325 variants across 16 exercises in 6 programming languages — using `azure-openai-gpt-5-mini` at `reasoning_effort=medium`. This configuration matches the Artemis production deployment at the time of evaluation. Each run derives its identifier from the Artemis git branch and commit hash; the code snapshot ZIP stored alongside the results captures the exact source files and prompt templates active during the run.

Each V1 configuration executes three runs per variant to reduce the influence of LLM non-determinism on the reported metrics; V2 configurations execute one run each, which Section 8.8 addresses as a limitation of the current evaluation scope.

#### 8.1.2 Metrics and Evaluation Criteria

The evaluation computes precision, recall,  $F_1$ , Span  $F_1$ , IoU, execution time, and cost per run.  $F_1$  serves as the primary ranking metric. Span  $F_1$  and IoU measure

how accurately the pipeline localizes issues within artifacts. Issue matching uses a greedy algorithm that pairs predicted and gold issues by category, maximizing Span  $F_1$  across all valid pairs; unmatched predictions count as false positives, and unmatched gold issues count as false negatives.

The primary improvement criterion is higher precision and  $F_1$  relative to the baseline, subject to recall remaining above 0.75. A recall below this threshold means the system misses more than one quarter of genuine inconsistencies, which is insufficient for reliable exercise quality assurance.

### 8.1.3 Research Questions

The evaluation addresses three research questions derived from the limitations of the existing system identified in Section 2.6.

**RQ1: Precision improvement.** Does the detect-then-verify pipeline reduce false positives on the extended dataset without a proportional loss in recall? This question targets the core limitation of the baseline system — low precision — and evaluates whether a post-detection verification step changes the precision-recall trade-off in a direction that makes the report actionable for instructors.

**RQ2: Multi-language generalization.** Does the language-aware context filter improve detection quality on non-Java exercises compared to the baseline, which applies no language-specific filtering? This question evaluates whether the strategy pattern introduced in the filter service produces measurable per-language improvements on the Assembler, C, Python, SQL, and Swift exercises in V2.

**RQ3: Combined approach.** When the language-aware filter and the verification checker run together, does the combined configuration achieve a higher  $F_1$  than the filter alone? This question evaluates whether the two modifications complement each other or whether one subsumes the benefit of the other. The verification checker evaluation in Section 8.4 runs on top of the language-aware context filter in the Artemis production branch, so its reported numbers already reflect the combined effect of both contributions. An ablation that evaluates the verification checker without the filter remains open as future work.

## 8.2 Baseline Approach

The baseline approach represents the Artemis production system at the start of this thesis. It defines the starting point against which all subsequent approaches measure their improvement and establishes that the Artemis pipeline is equivalent to the published PECV-bench reference implementation.

**Design.** The baseline corresponds to the Artemis production system at the time of evaluation, identified by commit 66c9e6b98d. Figure 11 shows the pipeline flow. The pipeline renders the problem statement, template repository, and solution repository into a single line-numbered text block without language-specific file filtering. The rendered context passes in parallel to two LLM checkers: a structural checker targeting `METHOD_RETURN_TYPE_MISMATCH`, `METHOD_PARAMETER_MISMATCH`, `CONSTRUCTOR_PARAMETER_MISMATCH`, `ATTRIBUTE_TYPE_MISMATCH`, and `VISIBILITY_MISMATCH`; and a semantic checker targeting `IDENTIFIER_NAMING_INCONSISTENCY`. The pipeline merges the issues returned by both checkers and forwards them directly to the caller without any post-detection verification step. The design mirrors the PECV-bench reference pipeline [Die+25], which runs the same structural and semantic checkers in parallel.

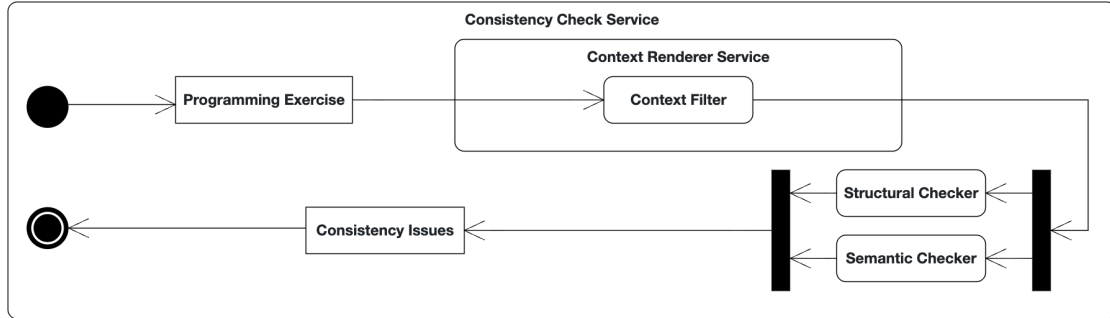


Figure 11: Activity diagram of the baseline consistency check pipeline [Die+25].

**Results.** On V1, the baseline achieves precision 0.555, recall 0.957, and  $F_1$  0.703 across 3 runs. These numbers are consistent with the reference pipeline reported in [Die+25] (precision 0.571, recall 0.939,  $F_1$  0.710), confirming that the Artemis implementation is equivalent to the published baseline.



| Approach       | TP  | FP  | FN | Precision | Recall | F <sub>1</sub> |
|----------------|-----|-----|----|-----------|--------|----------------|
| artemis-HEAD   | 267 | 214 | 12 | 0.555     | 0.957  | 0.703          |
| pecv-reference | 262 | 197 | 17 | 0.571     | 0.939  | 0.710          |

Table 5: Baseline aggregate results on V1 (91 variants, 3 runs). The artemis-HEAD approach corresponds to the Artemis production system; pecv-reference is the published PECV-bench baseline [Die+25].

On V2, precision drops to 0.531, recall to 0.785, and F<sub>1</sub> to 0.633 (260 TP, 230 FP, 71 FN). Per-exercise results reveal substantial variation. Swift exercises perform above average (TC1-Bookstore F<sub>1</sub> 0.809, TC2-Developer F<sub>1</sub> 0.708), while SQL exercises perform poorly: TA1-SQL\_Advanced produces zero detections (0 TP, 0 FP, 18 FN), and TA2-SQL\_Basics reaches only F<sub>1</sub> 0.400. Quantum Computing Lab exercises cluster between F<sub>1</sub> 0.524 and 0.553. Java exercises from V1 retain comparable performance on V2 (F<sub>1</sub> 0.598–0.737).

| Exercise              | TP         | FP         | FN        | Prec.        | Rec.         | F <sub>1</sub> | Span F <sub>1</sub> | IoU          |
|-----------------------|------------|------------|-----------|--------------|--------------|----------------|---------------------|--------------|
| ERA2021 / H00-ASM     | 18         | 16         | 3         | 0.529        | 0.857        | 0.655          | 0.687               | 0.564        |
| ERA2021 / H03-Graf.   | 11         | 11         | 7         | 0.500        | 0.611        | 0.550          | 0.731               | 0.628        |
| ERA2021 / P01-Ray.    | 16         | 8          | 2         | 0.667        | 0.889        | 0.762          | 0.703               | 0.578        |
| IOS26 / TC1-Bookstore | 19         | 9          | 0         | 0.679        | 1.000        | 0.809          | 0.745               | 0.648        |
| IOS26 / TC2-Developer | 17         | 13         | 1         | 0.567        | 0.944        | 0.708          | 0.888               | 0.810        |
| ISE22 / H05E01-REST   | 15         | 11         | 3         | 0.577        | 0.833        | 0.682          | 0.593               | 0.473        |
| ISE22 / H10E01-Cont.  | 16         | 9          | 2         | 0.640        | 0.889        | 0.744          | 0.675               | 0.561        |
| ITP2425 / H01E01      | 28         | 18         | 2         | 0.609        | 0.933        | 0.737          | 0.445               | 0.318        |
| ITP2425 / H02E02      | 26         | 32         | 3         | 0.448        | 0.897        | 0.598          | 0.456               | 0.331        |
| ITP2425 / H05E01      | 32         | 31         | 2         | 0.508        | 0.941        | 0.660          | 0.443               | 0.308        |
| ITP2425 / SE01E01-UML | 17         | 12         | 1         | 0.586        | 0.944        | 0.723          | 0.625               | 0.490        |
| MTG26 / TA1-SQL-Adv.  | 0          | 0          | 18        | 0.000        | 0.000        | 0.000          | —                   | —            |
| MTG26 / TA2-SQL-Bas.  | 8          | 14         | 10        | 0.364        | 0.444        | 0.400          | 0.796               | 0.712        |
| QCSL25 / QC01         | 11         | 13         | 7         | 0.458        | 0.611        | 0.524          | 0.815               | 0.720        |
| QCSL25 / QC02         | 13         | 16         | 5         | 0.448        | 0.722        | 0.553          | 0.640               | 0.540        |
| QCSL25 / QC03         | 13         | 17         | 5         | 0.433        | 0.722        | 0.542          | 0.711               | 0.617        |
| <b>Total</b>          | <b>260</b> | <b>230</b> | <b>71</b> | <b>0.531</b> | <b>0.785</b> | <b>0.633</b>   | <b>0.625</b>        | <b>0.511</b> |

Table 6: Baseline per-exercise results on V2 (325 variants, 1 run). SQL and Quantum Computing Lab exercises show the lowest F<sub>1</sub> scores, driven by context rendering problems on non-Java artifacts.

**Discussion.** The V1-to-V2 precision drop from 0.555 to 0.531 is modest, but the recall drop from 0.957 to 0.785 indicates that the pipeline struggles with the structural diversity introduced by non-Java exercises. The complete failure on `TA1-SQL_Advanced` points to a context rendering problem: without language-aware filtering, the rendered context for SQL exercises includes files that dilute or obscure the relevant source content, causing the LLM to produce no structured output. The low  $F_1$  on Quantum Computing Lab exercises follows from a similar cause — the rendered context mixes Python test harnesses and domain-specific notation that the checkers are not prompted to handle. Java exercises maintain higher recall because the context renderer was originally designed around Java file structures. The 230 false positives on V2 represent the dominant quality problem: with a precision of 0.531, an instructor reviewing the report must discard roughly one in every two reported issues, which negates the time savings the system intends to provide.

**Limitations.** The baseline applies no language-specific filtering, causing irrelevant files to appear in the rendered context for non-Java exercises. It applies no verification step, so every issue reported by either checker reaches the instructor regardless of whether a second analysis pass would confirm it. Both limitations are addressed by the two approaches evaluated in the following sections.

### 8.3 Language-Aware Context Filter

The language-aware context filter addresses the first limitation of the baseline: the absence of language-specific file filtering for non-Java exercises. Figure 12 shows how the filter integrates into the consistency check pipeline.

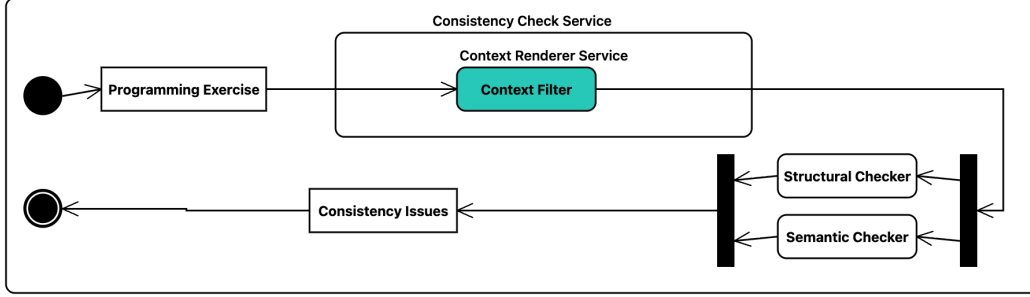


Figure 12: Integration of the language-aware context filter (■) into the consistency check pipeline. The filter applies four exclusion stages in sequence — language-specific patterns, global exclusion patterns, a binary safety net, and a size guard — before the filtered files reach the context renderer. Files that fail any stage do not enter the rendered context and therefore never reach the LLM.

**Design.** The language-aware context filter extends the context renderer with a pluggable `Strategy` interface registered in `HyperionProgrammingLanguageContextFilterService`. Each strategy combines global exclusion patterns — covering version control system metadata, build artifacts, binary file extensions, and IDE configuration files — with language-specific exclusions applied on top. Strategies are registered for six languages: Java excludes Gradle wrapper scripts, Maven wrappers, and Eclipse project files; Python excludes virtual environment directories and compiled bytecode; C excludes CMake build directories; Assembler and SQL rely on the global exclusions alone; Swift excludes the `.swiftpm` directory and `Package.resolved`. A default strategy applies only the global exclusions for any language without a registered strategy. After exclusion pattern matching, a binary safety net permits only files with extensions from a fixed allowlist of text-based formats, and files exceeding 100 KB drop before context rendering. The filtered file maps pass to the context renderer before prompt construction, reducing the rendered context to source-relevant content.

**Results.** Table 7 shows aggregate results on both dataset versions.

| Artemis Approach     | TP  | FP  | FN | Precision | Recall | F <sub>1</sub> |
|----------------------|-----|-----|----|-----------|--------|----------------|
| Baseline (V1)        | 267 | 214 | 12 | 0.555     | 0.957  | 0.703          |
| Language Filter (V1) | 260 | 169 | 19 | 0.606     | 0.932  | 0.734          |
| Baseline (V2)        | 260 | 230 | 71 | 0.531     | 0.785  | 0.633          |
| Language Filter (V2) | 256 | 204 | 75 | 0.557     | 0.773  | 0.647          |

Table 7: Aggregate results for the language-aware context filter compared to the baseline on V1 and V2.

On V1, the filter eliminates 45 false positives and improves precision from 0.555 to 0.606 and F<sub>1</sub> from 0.703 to 0.734, at the cost of a small recall drop from 0.957 to 0.932. On V2, the filter eliminates 26 false positives and improves precision from 0.531 to 0.557 and F<sub>1</sub> from 0.633 to 0.647. Recall remains comparable (0.785 vs. 0.773). Per-exercise improvements concentrate on exercises where the unfiltered context contained substantial noise: **H01E01-Lectures** precision rises from 0.609 to 0.690, **SE01E01-UML** achieves perfect recall with precision 0.692 (up from 0.586), and **H10E01-Containers** precision rises from 0.640 to 0.696. SQL exercises improve marginally — **TA1-SQL\_Advanced** recovers from zero detections to 3 TP — but remain the weakest exercises in the dataset (F<sub>1</sub> 0.162 and 0.350 respectively). Table 8 provides the full per-exercise breakdown.

| Exercise              | TP         | FP         | FN        | Prec.        | Rec.         | F <sub>1</sub> | Span F <sub>1</sub> | IoU          |
|-----------------------|------------|------------|-----------|--------------|--------------|----------------|---------------------|--------------|
| ERA2021 / H00-ASM     | 18         | 11         | 3         | 0.621        | 0.857        | 0.720          | 0.670               | 0.551        |
| ERA2021 / H03-Graf.   | 9          | 10         | 9         | 0.474        | 0.500        | 0.486          | 0.726               | 0.610        |
| ERA2021 / P01-Ray.    | 14         | 9          | 4         | 0.609        | 0.778        | 0.683          | 0.696               | 0.579        |
| IOS26 / TC1-Bookstore | 19         | 9          | 0         | 0.679        | 1.000        | 0.809          | 0.722               | 0.617        |
| IOS26 / TC2-Developer | 17         | 11         | 1         | 0.607        | 0.944        | 0.739          | 0.849               | 0.753        |
| ISE22 / H05E01-REST   | 14         | 8          | 4         | 0.636        | 0.778        | 0.700          | 0.623               | 0.494        |
| ISE22 / H10E01-Cont.  | 16         | 7          | 2         | 0.696        | 0.889        | 0.780          | 0.656               | 0.545        |
| ITP2425 / H01E01      | 29         | 13         | 1         | 0.690        | 0.967        | 0.806          | 0.388               | 0.264        |
| ITP2425 / H02E02      | 27         | 26         | 2         | 0.509        | 0.931        | 0.659          | 0.452               | 0.347        |
| ITP2425 / H05E01      | 31         | 24         | 3         | 0.564        | 0.912        | 0.697          | 0.446               | 0.317        |
| ITP2425 / SE01E01-UML | 18         | 8          | 0         | 0.692        | 1.000        | 0.818          | 0.635               | 0.507        |
| MTG26 / TA1-SQL-Adv.  | 3          | 16         | 15        | 0.158        | 0.167        | 0.162          | 0.723               | 0.625        |
| MTG26 / TA2-SQL-Bas.  | 7          | 15         | 11        | 0.318        | 0.389        | 0.350          | 0.838               | 0.753        |
| QCSL25 / QC01         | 10         | 10         | 8         | 0.500        | 0.556        | 0.526          | 0.862               | 0.799        |
| QCSL25 / QC02         | 12         | 14         | 6         | 0.462        | 0.667        | 0.545          | 0.676               | 0.561        |
| QCSL25 / QC03         | 12         | 13         | 6         | 0.480        | 0.667        | 0.558          | 0.705               | 0.624        |
| <b>Total</b>          | <b>256</b> | <b>204</b> | <b>75</b> | <b>0.557</b> | <b>0.773</b> | <b>0.647</b>   | <b>0.616</b>        | <b>0.504</b> |

Table 8: Language-aware context filter per-exercise results on V2 (325 variants, 1 run).

**Discussion.** The filter produces a consistent false positive reduction on both dataset versions — 45 fewer FP on V1 and 26 fewer FP on V2 — confirming that irrelevant files in the rendered context contribute to spurious issue reports. The precision gains are most pronounced on Java exercises with UML and architecture content, where the unfiltered context previously included non-source files. The near-zero improvement on SQL and Quantum Computing Lab exercises reflects a design limitation: the filter removes clearly irrelevant files but does not restructure the remaining context to make cross-artifact relationships explicit for non-Java languages. For SQL exercises, the rendered context still consists of flat `.sql` files without the class and method structure the structural checker targets, leaving the root cause of low recall unaddressed. The token-to-F<sub>1</sub> correlation on V2 is negligible ( $r = -0.018$ ,  $p = 0.752$ ), indicating that context size alone does not explain the variance in detection quality across variants.

**Limitations.** The filter strategy for each language is a static glob-pattern list defined at service initialization. Exercises that deviate from expected directory conventions — for example, Assembler exercises with non-standard file layouts — may still produce over-populated or under-populated contexts that the fixed patterns do not handle. The safety net extension allowlist is broad by design and permits any file with a recognized code extension regardless of whether it is relevant to the consistency check. SQL and Assembler exercises remain problematic not because the filter passes wrong files, but because the structural checker prompts assume object-oriented source structures that these languages do not provide. Addressing this requires language-specific prompt adaptation, not only file filtering, which falls outside the scope of this approach.

## 8.4 Verification Checker

The verification checker addresses the second limitation of the baseline: the absence of any post-detection filtering stage. It introduces a sequential third pass that re-evaluates each candidate issue before the report reaches the instructor. Figure 13 shows how the checker integrates into the pipeline.

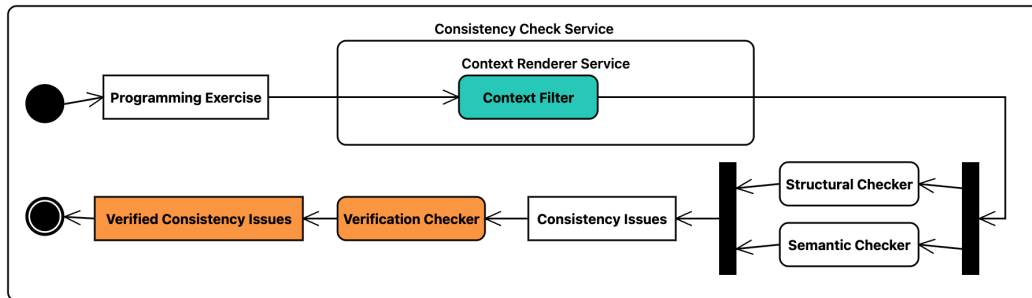


Figure 13: Activity diagram of the consistency check pipeline with the verification checker (■). The verification checker receives the merged candidate issue list, re-evaluates each candidate against the full rendered exercise context, and removes false positives before the final report reaches the instructor. When the verification call fails, the pipeline falls back to the pre-verification combined issue list.

**Design.** The verification checker extends the two-checker pipeline with a sequential third pass executed after the structural and semantic checkers complete. The two parallel checkers run concurrently; the pipeline serializes their combined output to JSON and injects it into the verification prompt as `detected_issues_json`. The verification prompt instructs the LLM to act as an educational quality assurance engineer: it receives the full rendered exercise context alongside the candidate issue list and decides, for each issue, whether it represents a genuine inconsistency, a false positive, or a duplicate. The verifier may also correct artifact locations, refine descriptions, and reclassify categories. The verifier uses a `UnifiedConsistencyIssues` structured output schema that accepts all six ontology categories, unlike the type-restricted schemas of the structural and semantic checkers. When the verification call fails at runtime, the service falls back to the pre-verification combined issue list, which preserves the structural and semantic results rather than aborting the analysis.

The verification step adds one LLM call per consistency check invocation. The pipeline aggregates token usage from all three calls. The average latency on V2 increases from 21 seconds per variant (language filter alone) to 44 seconds, and average cost per run rises from €0.012 to €0.018, reflecting the additional call.

**Results.** Table 9 shows aggregate results on both dataset versions.

| Approach                  | TP  | FP  | FN | Precision | Recall | F <sub>1</sub> |
|---------------------------|-----|-----|----|-----------|--------|----------------|
| Baseline (V1)             | 267 | 214 | 12 | 0.555     | 0.957  | 0.703          |
| Verification Checker (V1) | 259 | 31  | 20 | 0.893     | 0.928  | 0.910          |
| Baseline (V2)             | 260 | 230 | 71 | 0.531     | 0.785  | 0.633          |
| Verification Checker (V2) | 254 | 56  | 77 | 0.819     | 0.767  | 0.793          |

Table 9: Aggregate results for the verification checker compared to the baseline on V1 and V2.

On V1, the verification checker reduces false positives from 214 to 31 — an 85% reduction — and raises precision from 0.555 to 0.893 and F<sub>1</sub> from 0.703 to 0.910, while recall drops modestly from 0.957 to 0.928. On V2, false positives drop from 230 to 56 (76% reduction), precision rises from 0.531 to 0.819, and F<sub>1</sub> rises from 0.633 to 0.793. Recall decreases from 0.785 to 0.767, remaining above the 0.75 threshold. Several exercises reach zero false positives:

H05E01-Space\_Seal\_Farm, H10E01-Containers, H05E01-REST\_Architectural\_Style, TC1-Bookstore, TC2-Developer, and SE01E01-UML all report 0 FP. SQL exercises remain the weakest group: TA1-SQL\_Advanced drops to  $F_1$  0.062 (1 TP, 13 FP, 17 FN), and TA2-SQL\_Basics reaches  $F_1$  0.333. Table 10 provides the full per-exercise breakdown.

| Exercise              | TP         | FP        | FN        | Prec.        | Rec.         | $F_1$        | Span $F_1$   | IoU          |
|-----------------------|------------|-----------|-----------|--------------|--------------|--------------|--------------|--------------|
| ERA2021 / H00-ASM     | 18         | 2         | 3         | 0.900        | 0.857        | 0.878        | 0.481        | 0.356        |
| ERA2021 / H03-Graf.   | 9          | 5         | 9         | 0.643        | 0.500        | 0.563        | 0.707        | 0.598        |
| ERA2021 / P01-Ray.    | 14         | 2         | 4         | 0.875        | 0.778        | 0.824        | 0.480        | 0.357        |
| IOS26 / TC1-Bookstore | 18         | 0         | 1         | 1.000        | 0.947        | 0.973        | 0.416        | 0.273        |
| IOS26 / TC2-Developer | 15         | 0         | 3         | 1.000        | 0.833        | 0.909        | 0.520        | 0.369        |
| ISE22 / H05E01-REST   | 15         | 0         | 3         | 1.000        | 0.833        | 0.909        | 0.417        | 0.297        |
| ISE22 / H10E01-Cont.  | 16         | 0         | 2         | 1.000        | 0.889        | 0.941        | 0.504        | 0.377        |
| ITP2425 / H01E01      | 29         | 2         | 1         | 0.935        | 0.967        | 0.951        | 0.513        | 0.364        |
| ITP2425 / H02E02      | 27         | 9         | 2         | 0.750        | 0.931        | 0.831        | 0.484        | 0.352        |
| ITP2425 / H05E01      | 31         | 0         | 3         | 1.000        | 0.912        | 0.954        | 0.515        | 0.370        |
| ITP2425 / SE01E01-UML | 18         | 0         | 0         | 1.000        | 1.000        | 1.000        | 0.528        | 0.399        |
| MTG26 / TA1-SQL-Adv.  | 1          | 13        | 17        | 0.071        | 0.056        | 0.062        | 0.667        | 0.500        |
| MTG26 / TA2-SQL-Bas.  | 6          | 12        | 12        | 0.333        | 0.333        | 0.333        | 0.719        | 0.655        |
| QCSL25 / QC01         | 12         | 4         | 6         | 0.750        | 0.667        | 0.706        | 0.506        | 0.431        |
| QCSL25 / QC02         | 13         | 4         | 5         | 0.765        | 0.722        | 0.743        | 0.465        | 0.349        |
| QCSL25 / QC03         | 12         | 3         | 6         | 0.800        | 0.667        | 0.727        | 0.442        | 0.349        |
| <b>Total</b>          | <b>254</b> | <b>56</b> | <b>77</b> | <b>0.819</b> | <b>0.767</b> | <b>0.793</b> | <b>0.501</b> | <b>0.373</b> |

Table 10: Verification checker per-exercise results on V2 (325 variants, 1 run).

**Discussion.** The verification checker produces the most substantial improvement of all three approaches evaluated in this thesis. On V2, false positives fall from 230 to 56, precision rises by 0.288 absolute points, and  $F_1$  rises by 0.160 points over the baseline, while recall remains above the 0.75 threshold. The result confirms RQ1: the detect-then-verify pipeline reduces false positives without proportional recall loss.

The exercises that benefit most are those with clean object-oriented structure — Java and Swift — where the verifier can cross-reference candidate issues against the rendered source code with high reliability. Five exercises achieve perfect preci-



sion (0 FP), three of which also maintain recall above 0.88. For well-structured exercises, a single verification pass eliminates nearly all spurious reports.

SQL exercises expose a different failure mode. The verifier does not improve SQL results compared to the baseline — **TA1-SQL\_Advanced** worsens from 0 FP to 13 FP, indicating that the verifier itself generates false positives when operating on SQL content. The root cause matches the language filter analysis: the prompts assume object-oriented constructs that SQL exercises do not contain. The Span  $F_1$  scores for SQL (0.667–0.719) are among the highest in the dataset despite low  $F_1$ , indicating that when the verifier does match a gold issue, the artifact location is accurate — the problem is incorrect issue generation, not localization. The token-to- $F_1$  correlation on V2 is negligible ( $r = 0.018$ ,  $p = 0.746$ ), consistent with the baseline and filter results.

**Limitations.** The verification step adds approximately 23 seconds of latency per variant on V2 (44 s vs. 21 s for the language filter alone), which increases the time an instructor waits for a result. The additional LLM call increases per-run cost by approximately 50% compared to the language filter. The verifier operates on the same flat text representation as the two-checker pipeline and therefore cannot resolve issues that arise from missing structural context. SQL exercises expose a specific weakness: the verifier worsens precision by generating additional false positives from the unstructured SQL content, indicating that the system may require language-specific verification behavior. The verification prompt carries no programming-language parameter, so the same instructions apply regardless of whether the exercise targets Java, Assembler, or SQL.

## 8.5 Unified Consistency Checker

The unified consistency checker evaluates whether replacing the two parallel checkers — structural and semantic — with a single combined checker produces comparable or better detection quality. The motivation is to test whether prompt decomposition into two separate checkers provides measurable benefit over a single checker that targets all six inconsistency subcategories simultaneously. Figure 14 shows the pipeline flow.

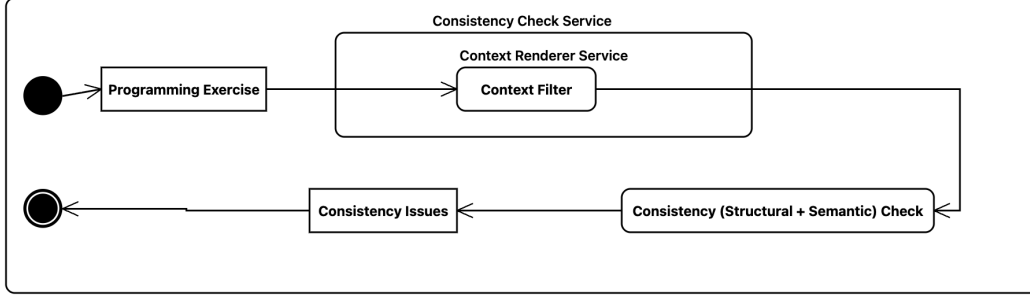


Figure 14: Activity diagram of the unified consistency checker pipeline. The pipeline applies the language-aware context filter and passes the rendered context to a single consistency checker that targets all six ontology subcategories — both structural and semantic — in one LLM invocation.

The checker returns the combined issue list directly without a verification step.

**Design.** The unified consistency checker replaces the parallel structural and semantic checkers with a single LLM invocation that targets all six inconsistency subcategories: `METHOD_RETURN_TYPE_MISMATCH`, `METHOD_PARAMETER_MISMATCH`, `CONSTRUCTOR_PARAMETER_MISMATCH`, `ATTRIBUTE_TYPE_MISMATCH`, `VISIBILITY_MISMATCH`, and `IDENTIFIER_NAMING_INCONSISTENCY`. The language-aware context filter applies before context rendering, identical to the filter-only configuration. No verification step runs after the single checker. The unified prompt instructs the LLM to act as a quality assurance engineer and to detect all structural and semantic contradictions across the problem statement, template repository, solution repository and test repository in a single pass. The `UnifiedConsistencyIssues` structured output schema constrains the LLM to the six ontology categories, matching the schema that the verification checker uses.

**Results.** Table 11 shows aggregate results on both dataset versions.

| Approach                         | TP  | FP  | FN | Precision | Recall | F <sub>1</sub> |
|----------------------------------|-----|-----|----|-----------|--------|----------------|
| Baseline (V1)                    | 267 | 214 | 12 | 0.555     | 0.957  | 0.703          |
| Unified Consistency Checker (V1) | 265 | 34  | 14 | 0.886     | 0.950  | 0.917          |
| Baseline (V2)                    | 260 | 230 | 71 | 0.531     | 0.785  | 0.633          |
| Unified Consistency Checker (V2) | 250 | 76  | 81 | 0.767     | 0.755  | 0.761          |

Table 11: Aggregate results for the unified consistency checker compared to the baseline on V1 and V2.

On V1, the unified checker reduces false positives from 214 to 34 — an 84% reduction — and raises precision from 0.555 to 0.886 and  $F_1$  from 0.703 to 0.917, while recall drops modestly from 0.957 to 0.950. On V2, false positives drop from 230 to 76 (67% reduction), precision rises from 0.531 to 0.767, and  $F_1$  rises from 0.633 to 0.761. Recall decreases from 0.785 to 0.755, remaining above the 0.75 threshold. Table 12 provides the full per-exercise breakdown on V2.

| Exercise              | TP         | FP        | FN        | Prec.        | Rec.         | $F_1$        | Span $F_1$   | IoU          |
|-----------------------|------------|-----------|-----------|--------------|--------------|--------------|--------------|--------------|
| ERA2021 / H00-ASM     | 16         | 2         | 5         | 0.889        | 0.762        | 0.821        | 0.690        | 0.577        |
| ERA2021 / H03-Graf.   | 5          | 9         | 13        | 0.357        | 0.278        | 0.313        | 0.847        | 0.755        |
| ERA2021 / P01-Ray.    | 16         | 1         | 2         | 0.941        | 0.889        | 0.914        | 0.652        | 0.528        |
| IOS26 / TC1-Bookstore | 18         | 0         | 1         | 1.000        | 0.947        | 0.973        | 0.698        | 0.601        |
| IOS26 / TC2-Developer | 18         | 1         | 0         | 0.947        | 1.000        | 0.973        | 0.812        | 0.714        |
| ISE22 / H05E01-REST   | 15         | 0         | 3         | 1.000        | 0.833        | 0.909        | 0.631        | 0.515        |
| ISE22 / H10E01-Cont.  | 16         | 2         | 2         | 0.889        | 0.889        | 0.889        | 0.605        | 0.482        |
| ITP2425 / H01E01      | 29         | 3         | 1         | 0.906        | 0.967        | 0.935        | 0.472        | 0.349        |
| ITP2425 / H02E02      | 28         | 7         | 1         | 0.800        | 0.966        | 0.875        | 0.435        | 0.322        |
| ITP2425 / H05E01      | 33         | 6         | 1         | 0.846        | 0.971        | 0.904        | 0.407        | 0.280        |
| ITP2425 / SE01E01-UML | 18         | 0         | 0         | 1.000        | 1.000        | 1.000        | 0.661        | 0.535        |
| MTG26 / TA1-SQL-Adv.  | 1          | 14        | 17        | 0.067        | 0.056        | 0.061        | 0.769        | 0.625        |
| MTG26 / TA2-SQL-Bas.  | 3          | 15        | 15        | 0.167        | 0.167        | 0.167        | 0.692        | 0.625        |
| QCSL25 / QC01         | 11         | 5         | 7         | 0.688        | 0.611        | 0.647        | 0.687        | 0.603        |
| QCSL25 / QC02         | 14         | 3         | 4         | 0.824        | 0.778        | 0.800        | 0.747        | 0.624        |
| QCSL25 / QC03         | 9          | 8         | 9         | 0.529        | 0.500        | 0.514        | 0.779        | 0.695        |
| <b>Total</b>          | <b>250</b> | <b>76</b> | <b>81</b> | <b>0.767</b> | <b>0.755</b> | <b>0.761</b> | <b>0.604</b> | <b>0.491</b> |

Table 12: Unified consistency checker per-exercise results on V2 (325 variants, 1 run).

**Discussion.** The unified consistency checker produces a substantial improvement over the baseline on both dataset versions. On V2, false positives drop from 230 to 76, precision rises by 0.236 absolute points, and  $F_1$  rises from 0.633 to 0.761, while recall stays above the 0.75 threshold. The result confirms that a single unified prompt targeting all six subcategories detects genuine inconsistencies with comparable reliability to the decomposed two-checker design.

Compared to the verification checker on V2 (precision 0.819,  $F_1$  0.793), the unified checker achieves lower precision and  $F_1$  but higher recall (0.755 vs. 0.767

— difference is negligible). The verification checker still outperforms overall, which indicates that a dedicated post-detection filtering pass contributes precision gains that a single unified prompt alone cannot match. The unified checker does outperform the language filter alone ( $F_1$  0.647) without requiring a second LLM call, which makes it an attractive trade-off when latency and cost are the primary constraints.

The per-exercise pattern mirrors the verification checker results: Swift and Java exercises perform well, SQL exercises remain the weakest group ( $F_1$  0.061 and 0.167), and ERA2021/H03-Grafikspeicher drops to  $F_1$  0.313 — the weakest non-SQL result in this configuration. The SE01E01-UML exercise achieves perfect precision and recall ( $F_1$  1.000), matching the verification checker on this exercise.

**Limitations.** The unified checker combines all six subcategories into one prompt, which increases the complexity of the instruction the LLM must follow and may reduce the depth of reasoning it applies to each individual subcategory. The single-checker design removes the opportunity to apply different few-shot examples and reasoning frameworks for structural versus semantic categories. The V2 evaluation covers one run, which carries the same single-run variance limitation as the other V2 configurations. A natural extension evaluates the unified checker with the verification checker applied on top, which would test whether the unified detection prompt combined with a post-detection filtering pass yields different results than the two-checker-plus-verification configuration already evaluated.

## 8.6 Future Approaches

The three approaches described in this section address the root cause identified in Section 2.6: the pipeline represents exercise artifacts as flat unstructured text, leaving cross-artifact relationships implicit. Each approach introduces a structured context representation that makes these relationships explicit before the LLM analyzes them. The approaches are designed and partially implemented but fall outside the evaluation scope of this thesis; Chapter 9 identifies their evaluation as the primary direction for future work. Figure 15 shows how each approach integrates into the existing pipeline.

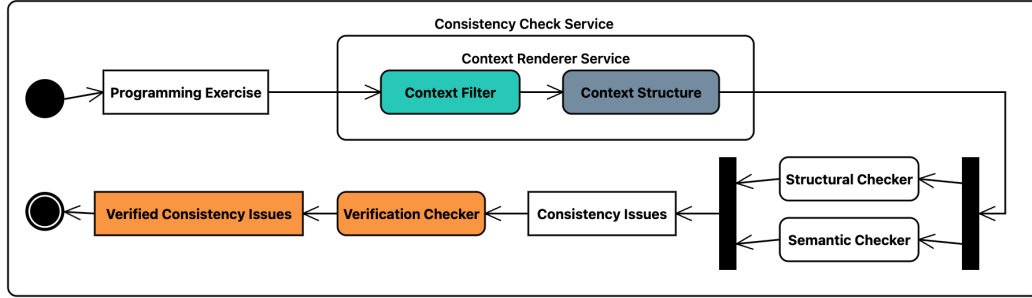


Figure 15: Integration of the three future approaches into the Consistency Check Service. Each approach injects a structured representation of the exercise artifacts into the context ( ) that the checkers receive.

### 8.6.1 Abstract Syntax Tree Representation

**Design.** The plain-text context renderer concatenates exercise artifacts as line-numbered source files, which leaves the structural relationships between code elements implicit. An LLM processing this representation must infer that a method declaration in the template repository corresponds to a requirement in the problem statement by reasoning over raw text alone. The Abstract Syntax Tree (AST) approach replaces this implicit inference with an explicit structural summary extracted directly from the source code.

The AST approach parses the filtered source files into abstract syntax trees before context rendering. The parser extracts the structural skeleton of each file — class declarations, method signatures with return types and parameter lists, attribute declarations with their types and visibility modifiers, and constructor signatures. The extracted elements map directly to the six inconsistency subcategories that the detection pipeline targets. The parser produces a compact structured summary per artifact that the context renderer injects alongside the plain-text content, giving the LLM an explicit inventory of the structural elements present in each repository. The AST summary for each repository appears as an additional section within the context block, positioned before the line-numbered source files, and the structural and semantic checker prompts reference this summary explicitly.

**Limitations and open questions.** The approach depends on a parser that handles the programming languages in the extended dataset — Java, Python, C, Assembler, SQL, and Swift. Each language requires a separate parsing strategy, and languages such as Assembler and SQL do not have conventional class or method structures that map cleanly to the ontology subcategories. The structured summary also increases the total token count of the rendered context, which may approach the context window limit for exercises with large codebases. A selective extraction strategy that includes only elements referenced in the problem statement could mitigate this overhead, but defining the selection criterion requires additional cross-artifact matching logic that is itself subject to the same ambiguity problem the approach aims to solve.

### 8.6.2 Knowledge Graph Representation

**Design.** The AST approach captures the structural properties of individual files but does not explicitly represent the relationships between artifacts — for example, that a method name in the problem statement refers to the same conceptual entity as a method declaration in the template repository. The knowledge graph approach models these cross-artifact relationships as a typed graph, making them available to the LLM as explicit relational context rather than implicit textual associations.

The graph schema defines nodes for four entity types: classes, methods, attributes, and problem statement requirements. Each node carries the attributes from its source artifact — for methods, the return type, parameter list, and visibility modifier; for requirements, the natural language description and the line range in the problem statement. Edges connect nodes across artifact boundaries through typed relations: a method node in the template repository connects to a requirement node in the problem statement through a *referenced-by* edge when both mention the same identifier; a method node in the solution repository connects to the corresponding template method node through an *implements* edge. The graph construction merges the AST-extracted elements from the template and solution repositories with the requirements from the problem statement. The consistency check prompt then queries the graph for requirement–implementation

pairs where the structural properties diverge, replacing the current approach of asking the LLM to discover these pairs through full-text reasoning.

**Limitations and open questions.** Graph construction requires reliable entity resolution — the system must determine that `calculateAverage` in the problem statement and `calculateAverage` in the template repository refer to the same entity despite appearing in different artifacts with different surrounding context. Exact string matching handles straightforward cases but fails when the problem statement uses natural language paraphrases or abbreviated names. The graph schema must also accommodate the structural diversity of the extended dataset: SQL exercises do not have classes or methods in the conventional sense, and Assembler exercises express structure through labels and register conventions rather than typed declarations. Extending the schema to cover these languages without losing the cross-artifact relationship model remains an open design problem.

### 8.6.3 Graph-Based Retrieval-Augmented Generation

**Design.** The knowledge graph approach provides complete relational coverage of an exercise but delivers all of this information to the LLM at once, which may introduce noise for exercises where only a subset of the relationships is relevant to the detected issue. Graph-based Retrieval-Augmented Generation (GraphRAG) addresses this by querying the knowledge graph selectively: instead of injecting the full graph into the prompt, the system retrieves only the subgraph most relevant to each checker invocation.

The retrieval step operates after context rendering and before prompt construction. For the structural checker, the retrieval query targets all requirement–method and requirement–attribute pairs where the structural properties differ between artifacts. For the semantic checker, the query targets all requirement nodes whose identifier appears under a different name in the template or solution repository. The retrieved subgraph — a compact set of nodes and typed edges — replaces or supplements the plain-text context in the checker prompt, focusing the LLM’s reasoning on the cross-artifact relationships most likely to contain genuine

inconsistencies. GraphRAG adds a fourth section to the context block containing the retrieved subgraph serialized as structured text, positioned before the line-numbered source files.

**Limitations and open questions.** The retrieval quality depends directly on the completeness of the knowledge graph: when the graph construction misses an entity or misclassifies a relationship, the retrieval step cannot surface it, and the checker operates without evidence for that inconsistency. This creates a dependency chain — GraphRAG inherits all limitations of the knowledge graph approach and adds retrieval-specific failure modes on top. Defining a retrieval strategy that is precise enough to reduce noise without discarding relevant relationships requires empirical tuning against the annotated dataset, which falls outside the evaluation scope of this thesis. The interaction between the retrieved subgraph and the plain-text source files in the prompt needs careful prompt engineering to avoid contradictory signals when the graph and the source text disagree due to construction errors.

## 8.7 Overall Discussion

The three approaches evaluated in this chapter address the false positive problem at different levels of the pipeline. The language-aware context filter reduces noise at the input level by removing irrelevant files before rendering; the verification checker reduces noise at the output level by filtering the combined issue list through a second LLM pass. Together, they reduce false positives from 230 to 56 on V2 and raise  $F_1$  from 0.633 to 0.793 relative to the baseline, while keeping recall above the 0.75 threshold. The unified consistency checker evaluates whether prompt consolidation achieves comparable precision gains to the two-checker design without a verification step, reaching  $F_1$  0.761 on V2 — above the language filter alone (0.647) but below the verification checker (0.793).

The results confirm RQ1: the detect-then-verify pipeline achieves a substantial precision improvement ( $0.531 \rightarrow 0.819$ ) without proportional recall loss ( $0.785 \rightarrow 0.767$ ). The 76% false positive reduction means an instructor reviewing the report now encounters roughly one spurious issue for every four genuine ones,



compared to nearly one in two for the baseline. The results partially confirm RQ2: the language filter improves precision on most non-Java exercises, but SQL exercises remain the weakest group across all three approaches. The filter removes irrelevant files but does not address the structural mismatch between the ontology categories and SQL semantics. The results address RQ3 through a layered configuration: the verification checker evaluation runs on top of the language-aware context filter in the Artemis production branch, so the reported verification checker numbers already reflect the combined effect of both contributions. The combined configuration achieves an  $F_1$  of 0.793 on V2, which exceeds both the filter-only  $F_1$  of 0.647 and the baseline  $F_1$  of 0.633. The unified consistency checker provides a complementary data point: it achieves  $F_1$  0.761 on V2 without a verification step, which isolates the contribution of prompt consolidation from post-detection filtering and confirms that the verification pass in the two-checker pipeline accounts for the remaining precision gap between the unified checker and the verification checker. An ablation run that evaluates the verification checker without the filter remains open as future work and would quantify the marginal contribution of each component independently.

Two persistent failure modes emerge from the per-exercise analysis. First, SQL exercises fail across all approaches: the ontology defines these categories — method return type, parameter, constructor, attribute type, visibility, and identifier naming — around object-oriented constructs that do not map to SQL query structure. The verifier worsens the SQL results relative to the language filter by generating additional false positives from the unstructured SQL content. Second, Span  $F_1$  degrades in the verification checker results compared to the baseline and language filter (0.501 vs. 0.616 and 0.630): the verifier correctly identifies which issues are genuine but reports artifact locations based on LLM interpretation of line-numbered flat text rather than structurally extracted spans. Both failure modes point to the same root cause identified in Section 2.6: the pipeline represents exercise artifacts as flat text, forcing the LLM to infer structure, cross-artifact relationships, and precise locations from raw content. The three future approaches

described above address this root cause directly by introducing structured context representations before LLM analysis.

## 8.8 Threats to Validity

**Internal validity.** The gold-standard annotations in the V2 dataset came from executing the baseline consistency check pipeline on each materialized variant with manual verification of every annotation. Variants where the baseline failed to detect the injected inconsistency were identified and corrected during verification. The annotation process relied on a single annotator, which introduces the risk of systematic bias in category assignment and location precision. Inconsistency types that are systematically ambiguous under the ontology — for example, cases where both `METHOD_PARAMETER_MISMATCH` and `IDENTIFIER_NAMING_INCONSISTENCY` could apply — may carry a consistent label under one category, which would penalize a checker that detects the correct inconsistency under the alternative label. Chapter 7 describes the four safeguards that limit detector-induced bias; a second independent annotator on a sampled subset would provide an inter-annotator agreement estimate and remains open as future work.

**Construct validity.** The evaluation metrics measure detection performance against single-issue synthetic variants. Real exercises may contain multiple simultaneous inconsistencies of varying severity, and the current dataset design does not capture the interaction between co-occurring issues. The recall threshold of 0.75 that the evaluation uses as an acceptance criterion operationalizes pedagogical acceptability but does not derive empirically from instructor feedback. A user study with instructors would provide empirical grounding for this threshold.

**External validity.** All 16 exercises come from a single university. The results may not generalize to exercises from other institutions, pedagogical styles, or programming languages not represented in the dataset. The evaluation uses a single model configuration (`azure-openai-gpt-5-mini`, `reasoning_effort=medium`); results for other models or reasoning effort settings may differ. The V2 dataset covers six languages, but per-language exercise counts are uneven — four Java exercises, three Assembler exercises, two SQL exercises, two Swift exercises, and

one Python exercise — which makes language-level conclusions for underrepresented languages unreliable.

**Reliability.** LLM outputs are non-deterministic. V1 results average over three runs per configuration, which makes the V1 numbers reasonably robust against run-to-run variance. V2 results for the language filter, the verification checker, and the unified consistency checker each come from a single run, which is a clear limitation: single-run numbers carry higher variance than multi-run averages, and the thesis cannot attach statistical significance claims to the V2 differences reported here. The per-variant  $F_1$  correlation analysis with input token count ( $r = -0.018$  to  $0.046$  across approaches, all  $p > 0.4$ ) confirms that context size does not systematically explain per-variant  $F_1$  variance, but this analysis does not substitute for multi-run averages when comparing aggregate metrics between configurations. Three independent runs per configuration on V2 would produce mean-with-dispersion estimates and enable paired statistical testing; Chapter 9 lists this as a primary direction for future work.

## 9 Conclusions & Future Work

This chapter summarizes the contributions of the thesis, discusses the significance of the results, reports the status of realized and open goals, and identifies directions for future work.

### 9.1 Summary

Inconsistencies across the artifacts of multi-artifact programming exercises create extraneous cognitive load for students and undermine assessment fairness. The existing LLM-based detection system in Artemis recovers most real inconsistencies but reports a high number of false positives, which forces instructors to manually verify each finding and erodes trust in the automated analysis. This thesis addressed the precision bottleneck through three contributions.

The first contribution established a reproducible on-demand benchmarking workflow that connects the Artemis platform with the PECV-bench evaluation

framework. The workflow enables any developer to evaluate a detection approach against annotated exercise variants under identical conditions and produces versioned, traceable results. The second contribution extended the evaluation dataset from three Java exercises with 91 variants to 16 exercises with 325 variants across six TUM courses and six programming languages. This extension eliminates the single-language bias of the original dataset and enables cross-language evaluation of detection performance. The third contribution introduced a language-aware context filter and a verification checker into the detection pipeline. The context filter removes irrelevant files before context rendering, and the verification checker re-evaluates each detected issue to filter false positives. The combination of these two extensions raises precision from 0.531 to 0.819, reduces false positives by 75% on V2 dataset, and improves the  $F_1$  score from 0.633 to 0.793 on the extended dataset.

The central finding of this thesis is that a dedicated post-detection filtering stage improves detection precision without requiring changes to the underlying reasoning model. The verification checker demonstrates that decomposing the detection task into a detect-then-verify pipeline yields more reliable results than a single detection pass. The benchmarking workflow and the extended dataset provide the infrastructure for future researchers to build on these results and evaluate new detection approaches under comparable conditions.

## 9.2 Discussion

The precision improvement from 0.531 to 0.819 demonstrates that a dedicated post-detection filtering stage addresses the root cause of false positives more directly than prompt engineering alone. The verification checker succeeds because it operates with access to the full detection output — it can identify patterns across the combined structural and semantic findings that a single-pass checker cannot observe. The maintained recall of 0.767 confirms that the verifier does not over-filter: it removes spurious detections without suppressing genuine inconsistencies at a significant rate.

The multi-language evaluation reveals that language-aware context filtering consistently reduces false positives across all six programming languages in the extended dataset. The reduction of irrelevant files before context rendering confirms that the precision ceiling in the baseline is partly attributable to noise in the rendered context rather than solely to reasoning limitations of the LLM. The 56 remaining false positives — even after applying both extensions — suggest that the unstructured text representation remains a binding constraint — the LLM still resolves cross-artifact ambiguities implicitly, and structured context representations represent the most direct path toward further precision gains.

The evaluation relies on synthetic perturbations that inject one inconsistency per variant into clean base exercises. This controlled setup enables precise metric computation but does not capture the full distribution of organic inconsistencies that instructors introduce during real course development. The statistical significance of per-language conclusions is limited by the number of exercises per language — two to three exercises per non-Java language — which means that language-specific results should be interpreted as directional rather than conclusive.

### 9.3 Status

Table 13 summarizes the status of the fifteen functional requirements defined in Section 4.2. The table indicates which requirements have been fully realized (●), partially realized (◐), and remain open (○). The following sections provide a detailed discussion of the realized and open goals.

| Requirement | Title                                               | Status |
|-------------|-----------------------------------------------------|--------|
| FR1         | Analyze Detection Performance                       | ●      |
| FR2         | Connect Benchmark to Production System              | ●      |
| FR3         | Execute Benchmark Evaluation                        | ●      |
| FR4         | Compare Approaches Against Baseline                 | ●      |
| FR5         | Manage Benchmark Dataset                            | ●      |
| FR6         | Support Multiple Programming Languages in Dataset   | ●      |
| FR7         | Separate Public and Confidential Components         | ●      |
| FR8         | Improve Inconsistency Detection Precision           | ●      |
| FR9         | Verify Detected Issues                              | ●      |
| FR10        | Localize Issues Accurately                          | ●      |
| FR11        | Classify and Report Issues                          | ●      |
| FR12        | Support Multiple Programming Languages in Detection | ●      |
| FR13        | Render Exercise Context                             | ◐      |
| FR14        | Represent Exercise Structure                        | ○      |
| FR15        | Provide Structured Context to Analysis              | ○      |

Table 13: Implementation status of the functional requirements: fully realized (●), partially realized (◐), and open (○).

### 9.3.1 Realized Goals

The benchmarking workflow realizes the evaluation infrastructure objectives. Any developer can execute a full benchmark run against a running Artemis instance through a single entry point, with results automatically committed to the PECV-bench repository as a pull request (FR1–FR4). Each run records the exact Artemis codebase state through a git-derived approach identifier and a code snapshot of the Hyperion source files, ensuring that any evaluation run can be reproduced under identical conditions. The dataset management infrastructure supports versioned exercises across multiple dataset versions, with a public–private repository split that protects confidential course materials while keeping benchmark infrastructure and results openly accessible (FR5–FR7).

The dataset extension delivers 325 annotated variants across 16 exercises in 6 programming languages from 6 TUM courses, extending the original V1 dataset of 91 variants across 3 Java exercises (FR5, FR6). All annotations were manually verified for location accuracy and category correctness.

The two detection improvements are fully implemented and evaluated (FR8–FR12). The language-aware context filter applies language-specific exclusion strategies registered for all six programming languages in the extended dataset. The verification checker introduces a sequential third LLM pass that re-evaluates the combined structural and semantic checker output and removes false positives. On V2, the two extensions together raise precision from 0.531 to 0.819, reduce false positives from 230 to 56, and improve  $F_1$  from 0.633 to 0.793, while maintaining recall above 0.75.

### 9.3.2 Open Goals

FR13 is partially realized: the context renderer produces line-numbered plain-text content for the problem statement, template repository, and solution repository, but the structured representations — AST summaries and cross-artifact relationship sections — are designed and not yet integrated into the rendering pipeline.

FR14 and FR15 remain open. The design of the exercise structure domain model and the structured context injection mechanism is complete, as described in Section 8.5, but the implementation is pending. FR15 depends on FR14 and therefore shares the same open status. These two requirements represent the primary remaining implementation work and are identified as the first priority for future work in Section 9.4.

Three of the six ontology inconsistency categories — Assessment, Temporal, and Scope — have no corresponding detection prompts or evaluation data in the current implementation. Instantiating these categories requires new prompt designs and an expanded gold-standard dataset. The extended V2 dataset covers two to three exercises per non-Java language, which is insufficient for statistically reliable per-language conclusions; expanding the dataset with additional exercises per language remains an open task. A classroom pilot with real instructors has not been conducted; the reported precision improvements are supported by metric evidence only and have not been validated against practical instructor workflows.

## 9.4 Future Work

The immediate next step is to evaluate the designed context representation enhancements — abstract syntax trees, knowledge graphs, and graph-based retrieval-augmented generation — against the extended V2 dataset using the benchmarking workflow. These approaches provide the LLM with structured artifact relationships and have the potential to further reduce false positives by replacing implicit cross-artifact reasoning with explicit structural information. The benchmarking infrastructure from the first objective supports this evaluation without additional tooling.

The consistency ontology defines five categories, of which the current system instantiates only Structural and Semantic. Future work should design detection prompts for the Assessment, Temporal, and Scope categories and extend the gold-standard dataset with corresponding annotations. Covering these categories would broaden the scope of automated quality assurance beyond code-level contradictions to include pedagogical design issues.

A study where real instructors use the consistency check during exercise creation would validate whether the precision improvements lead to instructor trust and practical adoption. Such a study could combine quantitative metrics with qualitative instructor feedback on report usefulness, severity calibration, and workflow integration.

The dataset currently contains two to three exercises per non-Java language. Expanding the dataset with additional exercises from courses at TUM and other universities would strengthen per-language evaluation and test cross-institution generalizability. Evaluating open-weight models as alternatives to closed-source APIs would address the need for sovereign hosting at European universities. Exploring automated remediation — generating patches that fix detected inconsistencies with sandbox-based correctness verification — represents a natural extension from detection to correction.



## List of Figures

|          |                                                                                                                                                                                                                                                                              |    |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Figure 1 | Correlation between input tokens and per-variant $F_1$ score across all four evaluated reasoning models (o4-mini, Grok 3 Mini, Gemini 2.5 Flash, Gemini 2.5 Flash Lite) on the dataset with 91 perturbed variants. Each point represents a single variant evaluation. ....   | 3  |
| Figure 2 | Dependency structure of the four exercise artifacts in Artemis. ....                                                                                                                                                                                                         | 8  |
| Figure 3 | Consistency issue ontology for multi-artifact programming exercises. Each subcategory defines a specific type of inconsistency between the problem statement, template repository, solution repository and test repository [Die+25]. ....                                    | 9  |
| Figure 4 | Simplified example of the context rendering output format. The rendered context combines the problem statement as a pseudo-file with line numbers and the template repository with a file tree overview followed by line-numbered source files [Die+25]. ....                | 11 |
| Figure 5 | Activity diagram of the existing Hyperion consistency check pipeline. ....                                                                                                                                                                                                   | 12 |
| Figure 6 | Domain model of the consistency check system in Artemis. The <i>Benchmark</i> component in PECV-bench uses the <i>ConsistencyCheckReport</i> through a dependency relationship, reading the JSON output to evaluate detection approaches against ground-truth annotations. . | 27 |
| Figure 7 | Activity diagram of the benchmarking workflow across three actors: the developer, the Artemis platform, and                                                                                                                                                                  |    |

|           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |    |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
|           | PECV-bench. The developer selects a detection approach and triggers the benchmark run. ....                                                                                                                                                                                                                                                                                                                                                                                                | 29 |
| Figure 8  | Component diagram of the system architecture. Colours indicate the contribution status of each component: ■ new in this thesis, ■ extended in this thesis, ■ reused and identified as open for future work. ....                                                                                                                                                                                                                                                                           | 33 |
| Figure 9  | Activity diagram of the benchmarking workflow. The workflow starts after the developer boots a local Artemis application server and waits for it to become ready. ....                                                                                                                                                                                                                                                                                                                     | 38 |
| Figure 10 | End-to-end construction of the V2 dataset. Six university courses contribute raw exercises that the thesis author consolidates into a single working dataset. The thesis author identifies and fixes 126 pre-existing inconsistencies to produce a clean annotated dataset, then distributes the resulting base exercises across the public PECV-bench repository and the private PECV-bench-dataset repository according to the confidentiality arrangement each instructor accepts. .... | 47 |
| Figure 11 | Activity diagram of the baseline consistency check pipeline [Die+25]. ....                                                                                                                                                                                                                                                                                                                                                                                                                 | 54 |
| Figure 12 | Integration of the language-aware context filter (■) into the consistency check pipeline. The filter applies four exclusion stages in sequence — language-specific patterns, global exclusion patterns, a binary safety net, and a size guard — before the filtered files reach the context renderer. Files that fail any stage do not enter the rendered context and therefore never reach the LLM. ....                                                                                  | 57 |
| Figure 13 | Activity diagram of the consistency check pipeline with the verification checker (■). The verification checker receives the merged candidate issue list, re-evaluates each candidate                                                                                                                                                                                                                                                                                                       |    |

against the full rendered exercise context, and removes false positives before the final report reaches the instructor. When the verification call fails, the pipeline falls back to the pre-verification combined issue list. . . . . 60

**Figure 14** Activity diagram of the unified consistency checker pipeline. The pipeline applies the language-aware context filter and passes the rendered context to a single consistency checker that targets all six ontology subcategories — both structural and semantic — in one LLM invocation. The checker returns the combined issue list directly without a verification step. . . . . 64

**Figure 15** Integration of the three future approaches into the Consistency Check Service. Each approach injects a structured representation of the exercise artifacts into the context ( ■ ) that the checkers receive. . . . . 67

## List of Tables

|         |                                                                                                                                                                                                                                                                                                 |    |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Table 1 | Baseline results across four reasoning models on the PECV-bench dataset with 91 perturbed variants containing 93 annotated issues. The highlighted row (GPT o4-mini) achieves the highest $F_1$ score of 0.75 [Die+25]. . . . .                                                                 | 14 |
| Table 2 | Comparison of the PECV-bench reference pipeline and the Artemis Hyperion consistency check. The Artemis-HEAD pipeline runs on the develop branch (commit 66c9e6b <sup>10</sup> ). Both configurations use the <code>azure-openai-gpt-5-mini</code> model and original dataset [Die+25]. . . . . | 16 |
| Table 3 | Per-exercise variant counts in the V1 dataset, which contains the three Java exercises from the original PECV-bench benchmark published by Dietrich et al. [Die+25]. .                                                                                                                          | 45 |
| Table 4 | Per-exercise variant counts in the V2 dataset. The three ITP2425 exercises from V1 reappear here to represent the full dataset. . . . .                                                                                                                                                         | 45 |
| Table 5 | Baseline aggregate results on V1 (91 variants, 3 runs). The artemis-HEAD approach corresponds to the Artemis production system; pecv-reference is the published PECV-bench baseline [Die+25]. . . . .                                                                                           | 55 |
| Table 6 | Baseline per-exercise results on V2 (325 variants, 1 run). SQL and Quantum Computing Lab exercises show the lowest $F_1$ scores, driven by context rendering problems on non-Java artifacts. . . . .                                                                                            | 55 |
| Table 7 | Aggregate results for the language-aware context filter compared to the baseline on V1 and V2. . . . .                                                                                                                                                                                          | 58 |

---

<sup>10</sup><https://github.com/lslintum/Artemis/commit/66c9e6b98d>

|          |                                                                                                                            |    |
|----------|----------------------------------------------------------------------------------------------------------------------------|----|
| Table 8  | Language-aware context filter per-exercise results on V2<br>(325 variants, 1 run). . . . .                                 | 59 |
| Table 9  | Aggregate results for the verification checker compared to<br>the baseline on V1 and V2. . . . .                           | 61 |
| Table 10 | Verification checker per-exercise results on V2 (325 variants,<br>1 run). . . . .                                          | 62 |
| Table 11 | Aggregate results for the unified consistency checker<br>compared to the baseline on V1 and V2. . . . .                    | 64 |
| Table 12 | Unified consistency checker per-exercise results on V2 (325<br>variants, 1 run). . . . .                                   | 65 |
| Table 13 | Implementation status of the functional requirements: fully<br>realized (●), partially realized (◐), and open (○). . . . . | 76 |

## **Appendix A: Supplementary Material**

– Supplementary Material –

## Bibliography

- [KS18] S. Krusche and A. Seitz, “ArTEMiS: An Automatic Assessment Management System for Interactive Learning,” in *SIGCSE '18: Proceedings of the 49th ACM Technical Symposium on Computer Science Education*, Association for Computing Machinery, 2018.
- [Die+25] F. Dietrich, Y. Zhou, T. Wasner, S. Krusche, and M. Acosta, “LLM-Based Multi-Artifact Consistency Verification for Programming Exercise Quality Assurance,” 2025.
- [MM03] R. E. Mayer and R. Moreno, “Nine Ways to Reduce Cognitive Load in Multimedia Learning,” *Educational Psychologist*, vol. 38, no. 1, pp. 43–52, Jan. 2003.
- [Big96] J. Biggs, “Enhancing Teaching through Constructive Alignment,” *Higher Education*, vol. 32, no. 3, pp. 347–364, Oct. 1996.
- [Swe88] J. Sweller, “Cognitive Load During Problem Solving: Effects on Learning,” *Cognitive Science*, vol. 12, no. 2, pp. 257–285, 1988.
- [Swe94] J. Sweller, “Cognitive Load Theory, Learning Difficulty, and Instructional Design,” *Learning and Instruction*, vol. 4, no. 4, pp. 295–312, Jan. 1994.
- [Li+16] W. Li, J. H. Hayes, G. Antoniol, Y.-G. Guéhéneuc, and B. Adams, “Error leakage and wasted time: sensitivity and effort analysis of a requirements consistency checking process,” *Journal of Software: Evolution and Process*, vol. 28, no. 12, pp. 1061–1080, Oct. 2016.
- [Wei+22] J. Wei *et al.*, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2022.
- [Nus96] B. Nuseibeh, “To Be and Not to Be: On Managing Inconsistency in Software Development,” in *Proceedings of the 8th International Workshop on Software Specification and Design*, IEEE Computer Society Press, 1996, pp. 164–169.

- [SZ01] G. Spanoudakis and A. Zisman, “Inconsistency Management in Software Engineering: Survey and Open Research Issues,” 2001, *World Scientific Publishing Company*.
- [GKB11] A. Göknıl, I. Kurtev, and K. van den Berg, “Semantics of trace relations in requirements models for consistency checking and inferencing,” *Software & Systems Modeling*, vol. 10, no. 1, pp. 31–54, 2011.
- [KK16] S. Kim and D. Kim, “Automatic Identifier Inconsistency Detection Using Code Dictionary,” *Empirical Software Engineering*, vol. 21, no. 2, pp. 565–604, 2016.
- [PFL23] J. C. Paiva, Á. Figueira, and J. P. Leal, “Bibliometric Analysis of Automated Assessment in Programming Education: A Deeper Insight into Feedback,” *Electronics*, vol. 12, no. 10, 2023.
- [Kas+23] E. Kasneci *et al.*, “ChatGPT for Good? On Opportunities and Challenges of Large Language Models for Education,” *Learning and Individual Differences*, vol. 103, 2023.
- [Zha+24] Y. Zhao *et al.*, “CodeJudge-Eval: Can Large Language Models be Good Judges in Code Understanding?,” 2024. [Online]. Available: <https://arxiv.org/abs/2408.10718>
- [HWW25] R. Hu, X. Wang, and X.-C. Wen, “Benchmarking LLMs for Fine-Grained Code Review with Enriched Context in Practice,” 2025.
- [Bir+24] A. Birillo *et al.*, “One Step at a Time: Combining LLMs and Static Analysis to Generate Next-Step Hints for Programming Tasks,” in *Proceedings of the 24th Koli Calling International Conference on Computing Education Research*, ACM, 2024, pp. 1–12.
- [LZW25] L. Liu, W. Zhang, and L. Wei, “LLM-Powered Detection of Price Manipulation in DeFi,” 2025.
- [Wan+23] X. Wang *et al.*, “Self-Consistency Improves Chain of Thought Reasoning in Language Models,” in *The Eleventh International Conference on Learning Representations*, OpenReview.net, 2023.



- [Wen+23] Y. Weng *et al.*, “Large Language Models are Better Reasoners with Self-Verification,” in *Findings of the Association for Computational Linguistics: EMNLP 2023*, Association for Computational Linguistics, 2023, pp. 2550–2575.
- [Wan+20] W. Wang, G. Li, B. Ma, X. Xia, and T. Zhang, “Detecting Code Clones with Graph Neural Network and Flow-Augmented Abstract Syntax Tree,” in *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, IEEE, 2020, pp. 261–271.
- [Hog+21] A. Hogan *et al.*, “Knowledge Graphs,” *ACM Computing Surveys*, vol. 54, no. 4, pp. 1–37, 2021.
- [PC24] J. Z. Pan and o. Chen, “Unifying Large Language Models and Knowledge Graphs: A Roadmap,” 2024. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10387715>
- [ZC24] Z. Zhang and o. Chen, “GraphRAG: A Graph-Based Retrieval-Augmented Generation Method,” 2024. [Online]. Available: <https://arxiv.org/abs/2405.16506>
- [BD09] B. Bruegge and A. H. Dutoit, *Object Oriented Software Engineering Using UML, Patterns, and Java*. Prentice Hall, 2009.